

YOLOv8-Pose+RetinaFace 视觉检测实战（人体）

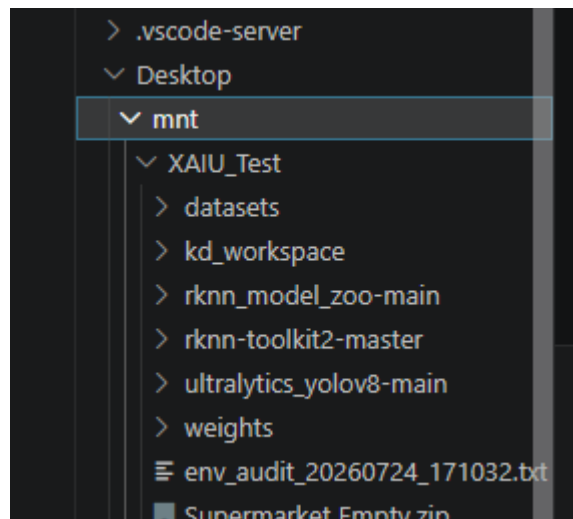
参考文献：

YOLOv8-Pose （追踪+行为分析）

解决：人在店内做了什么、去了哪里

demo

远程登陆改名



检查官方量化图片是否存在

代码块

```
1 cd /mnt/XAIU_Test/rknn_model_zoo
2
3 ls -lh datasets/COCO/coco_subset_20.txt
4
5 ls -lh datasets/COCO/subset
6
7 find datasets/COCO/subset \
8     -maxdepth 1 \
9     -type f \
10    \( -iname "*.jpg" -o -iname "*.png" \) \
11    | wc -l
```

```

(myconda) root@qm8b7M:~/Desktop/mnt/XAIU_Test# cd /mnt/XAIU_Test/rknn_model_zoo-main

ls -lh datasets/COCO/coco_subset_20.txt

ls -lh datasets/COCO/subset

find datasets/COCO/subset \
  -maxdepth 1 \
  -type f \
  \( -iname "*.jpg" -o -iname "*.png" \) \
  | wc -l
-rw-r--r-- 0 root 2000 519 Apr  9  2025 datasets/COCO/coco_subset_20.txt
total 0
-rw-r--r-- 0 root 2000 207K Apr  9  2025 000000005001.jpg
-rw-r--r-- 0 root 2000 209K Apr  9  2025 0000000038829.jpg
-rw-r--r-- 0 root 2000 150K Apr  9  2025 0000000052891.jpg
-rw-r--r-- 0 root 2000 102K Apr  9  2025 0000000075612.jpg
-rw-r--r-- 0 root 2000 14K Apr  9  2025 0000000098261.jpg
-rw-r--r-- 0 root 2000 202K Apr  9  2025 0000000181542.jpg
-rw-r--r-- 0 root 2000 234K Apr  9  2025 0000000215245.jpg
-rw-r--r-- 0 root 2000 242K Apr  9  2025 0000000277005.jpg
-rw-r--r-- 0 root 2000 231K Apr  9  2025 0000000288685.jpg
-rw-r--r-- 0 root 2000 81K Apr  9  2025 0000000301421.jpg
-rw-r--r-- 0 root 2000 136K Apr  9  2025 0000000334371.jpg
-rw-r--r-- 0 root 2000 113K Apr  9  2025 0000000348481.jpg
-rw-r--r-- 0 root 2000 281K Apr  9  2025 0000000373353.jpg
-rw-r--r-- 0 root 2000 272K Apr  9  2025 0000000397681.jpg
-rw-r--r-- 0 root 2000 153K Apr  9  2025 0000000414673.jpg
-rw-r--r-- 0 root 2000 166K Apr  9  2025 0000000419312.jpg
-rw-r--r-- 0 root 2000 110K Apr  9  2025 0000000465822.jpg
-rw-r--r-- 0 root 2000 103K Apr  9  2025 0000000475732.jpg
-rw-r--r-- 0 root 2000 203K Apr  9  2025 0000000559707.jpg
-rw-r--r-- 0 root 2000 110K Apr  9  2025 0000000574315.jpg
20

```

```
cd /root/yolov8_rk3588
```

```
python3 dual_yolov8_pose_rknn_camera.py \
```

```
--model /root/yolov8_rk3588/code/model/yolov8n-pose-int8.rknn \
```

```
--left /dev/video22 \
```

```
--right /dev/video31 \
```

```
--width 1920 \
```

```
--height 1080 \
```

```
--camera-fps 30 \
```

```
--display-fps 30 \
```

```
--input-size 640 \
```

```
--conf 0.50 \
```

```
--nms 0.40 \
```

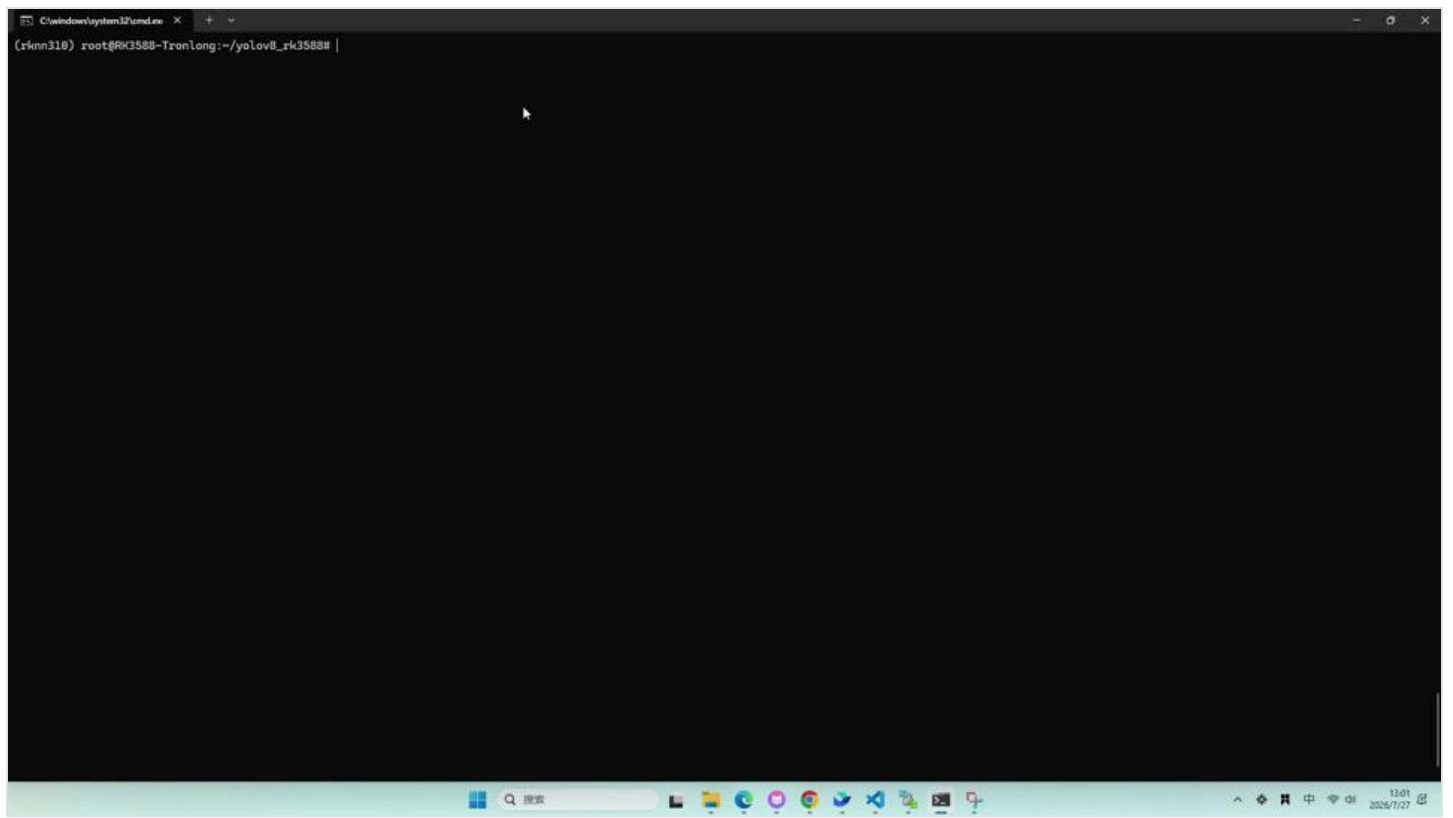
```
--kpt-conf 0.50 \
```

```
--core 012 \
```

```
--port 8080
```



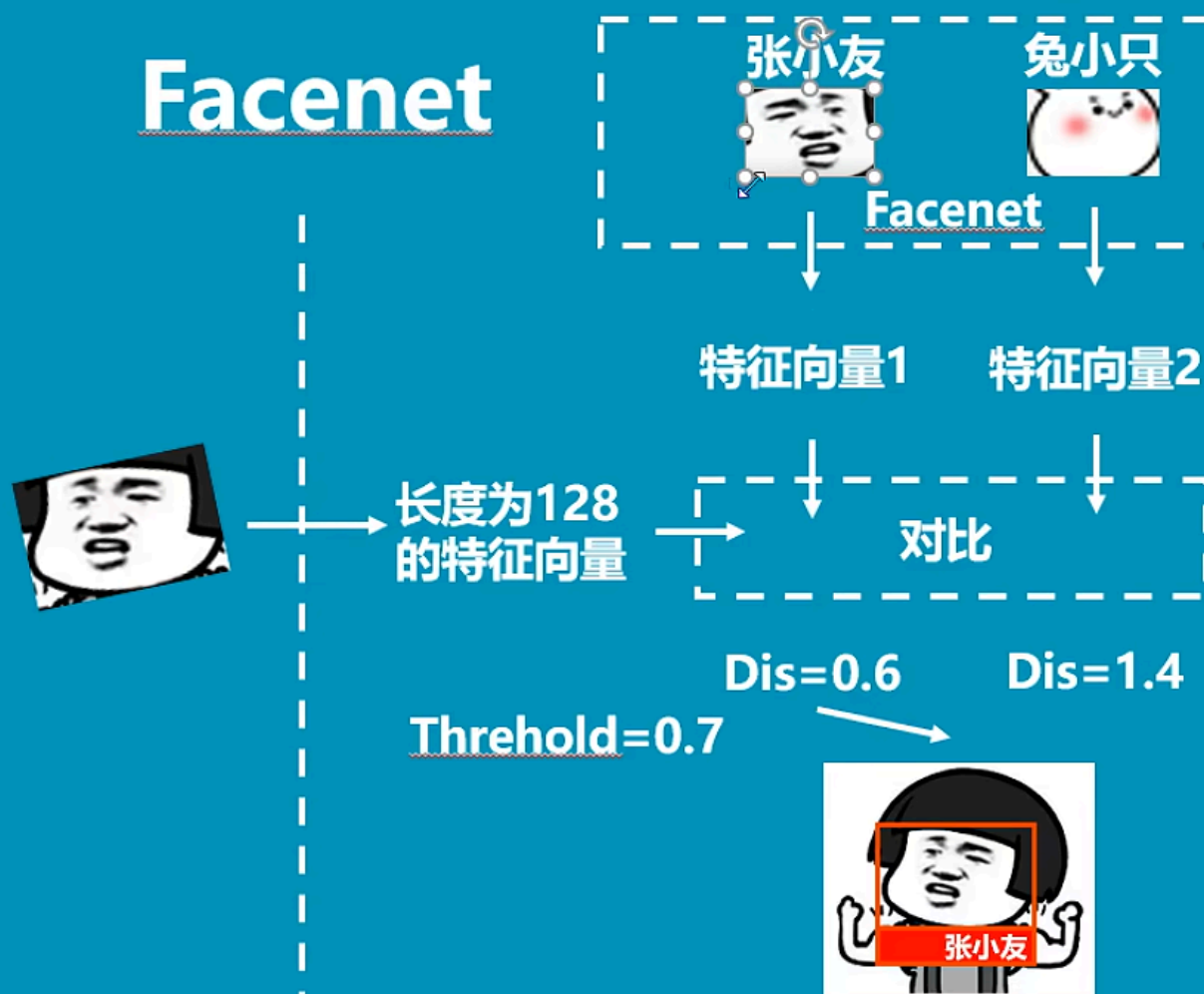
 dual_yolov8_pose_rknn_camera.py



RetinaFace （关键点+对齐）

Facenet

数据库



解决：入口处是否有人脸、人脸位置和姿态是否适合进一步识别

demo

量化

```
python convert.py \  
../model/RetinaFace_mobile320.onnx \  
rk3588 \  
i8 \  
../model/RetinaFace_mobile320-int8.rknn \  
2>&1 | tee convert_retinaface_int8.log
```

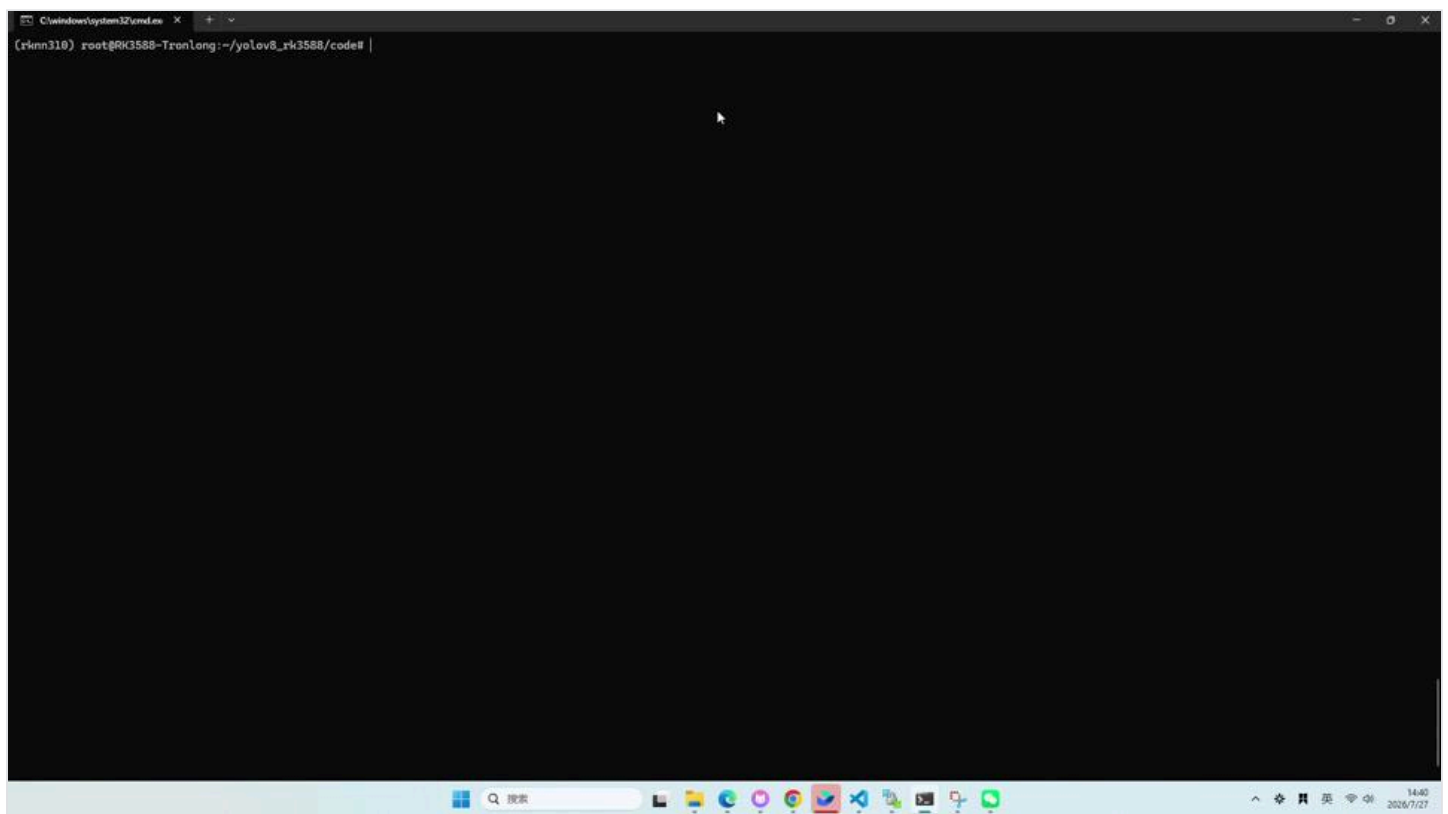


 realtime_retinaface_rknn_camera.py

```
source /root/yolov8_rk3588/rknn310/bin/activate  
cd /root/yolov8_rk3588/code  
python realtime_retinaface_rknn_camera.py \  
--model /root/yolov8_rk3588/code/model/RetinaFace_mobile320-int8.rknn \  
--left /dev/video22 \  
--right /dev/video31 \  

```

```
--width 1920 \  
--height 1080 \  
--camera-fps 30 \  
--display-fps 30 \  
--input-size 320 \  
--input-layout nhwc \  
--candidate-conf 0.02 \  
--conf 0.20 \  
--nms 0.50 \  
--core 012 \  
--port 8081
```



w600k_mbf（特征+识别）

demo

数据集洗练

https://figshare.com/articles/dataset/lfw-deepfunneled_reduced/7931432?file=14758619 (数据集5712张)

https://github.com/deepinsight/insightface/releases?utm_source=chatgpt.com (pt文件)

洗练条件：

每个一级文件夹代表一个人
↓
只允许图片数量 ≥ 4 张的人参与筛选
↓
选中某个人后，保留这个人的全部图片
↓
不随机抽取每人4张
↓
选择若干完整人员，使总图片数尽量接近100张
↓
所有图片复制到同一个目录

```
C:\Users\PC\Downloads>python "C:\Users\PC\Downloads\prepare_lfw_flat_quant.py" --root "C:\Users\PC\Downloads\lfw-deepfunneled_reduced_bkup\lfw-deepfunneled_reduced_bkup" --output "C:\Users\PC\Downloads\lfw_quant_flat_100" --min-images 4 --target-images 100 --seed 20260728 --dry-run
```

```
=====
源数据集: C:\Users\PC\Downloads\lfw-deepfunneled_reduced_bkup\lfw-deepfunneled_reduced_bkup
输出目录: C:\Users\PC\Downloads\lfw_quant_flat_100
人员文件夹总数: 5721
至少4张图片的合格人员: 606
目标图片数: 100
实际选择人员数: 9
实际选择图片数: 100
与目标差值: 0
=====
```

选中的人员（选中后保留该人的全部图片）：

000246_Anders_Fogh_Rasmussen	4张
000579_Binyamin_Ben-Eliezer	7张
001056_Condoleezza_Rice	11张
002124_Hillary_Clinton	14张
002425_Javier_Solana	10张
003716_Mian_Khursheed_Mehmood_Kasuri	4张
003997_Naomi_Watts	22张
004751_Roger_Moore	5张
004849_Saddam_Hussein	23张

当前是dry-run模式，没有复制任何图片。

```
C:\Users\PC\Downloads>
```

输出目录： C:\Users\PC\Downloads\lfw_quant_flat_100

人员文件夹总数： 5721

至少4张图片的合格人员： 606

目标图片数： 100

实际选择人员数： 9

实际选择图片数： 100

与目标差值： 0

选中的人员（选中后保留该人的全部图片）：

000246_Anders_Fogh_Rasmussen	4张
000579_Binyamin_Ben-Eliezer	7张
001056_Condoleezza_Rice	11张
002124_Hillary_Clinton	14张
002425_Javier_Solana	10张
003716_Mian_Khursheed_Mehmood_Kasuri	4张
003997_Naomi_Watts	22张
004751_Roger_Moore	5张
004849_Saddam_Hussein	23张

处理完成

复制人员数： 9

复制图片数： 100

平铺图片目录： C:\Users\PC\Downloads\lfw_quant_flat_100

人员清单： C:\Users\PC\Downloads\lfw_quant_flat_100\selected_people.csv

图片来源清单： C:\Users\PC\Downloads\lfw_quant_flat_100\image_manifest.csv

相对路径列表： C:\Users\PC\Downloads\lfw_quant_flat_100\dataset_relative.txt

量化命令（int_8+fp-100张）：

可以依次复制：

代码块

```
1 cd /root/Desktop/mnt/XAIU_Test
2
3 conda activate myconda
4
5 BASE=/root/Desktop/mnt/XAIU_Test
6
7 ONNX_MODEL="$(find "$BASE/weights" \
8   -maxdepth 1 \
9   -type f \
10  -name '*w600k_mbf.onnx' \
11  -print \
12  -quit)"
13
14 echo "ONNX_MODEL=$ONNX_MODEL"
15
16 test -s "$ONNX_MODEL" || {
17   echo "找不到w600k_mbf.onnx"
```

```

18     exit 1
19 }
20
21 mkdir -p \
22     "$BASE/face_recognition/calibration/lfw_112" \
23     "$BASE/face_recognition/rknn" \
24     "$BASE/face_recognition/logs"
25
26 python "$BASE/prepare_face_quant_112.py" \
27     --source "$BASE/lfw_quant_flat_100" \
28     --output "$BASE/face_recognition/calibration/lfw_112" \
29     --dataset "$BASE/face_recognition/calibration/dataset.txt" \
30     --size 112 \
31     --overwrite
32
33 set -o pipefail
34
35 python "$BASE/convert_w600k_mbf_to_rknn.py" \
36     --onnx "$ONNX_MODEL" \
37     --output "$BASE/face_recognition/rknn/w600k_mbf-fp.rknn" \
38     --mode fp \
39     --target rk3588 \
40     2>&1 | tee "$BASE/face_recognition/logs/convert_fp.log"
41
42 python "$BASE/convert_w600k_mbf_to_rknn.py" \
43     --onnx "$ONNX_MODEL" \
44     --output "$BASE/face_recognition/rknn/w600k_mbf-int8.rknn" \
45     --mode int8 \
46     --dataset "$BASE/face_recognition/calibration/dataset.txt" \
47     --target rk3588 \
48     2>&1 | tee "$BASE/face_recognition/logs/convert_int8.log"
49
50 ls -lh "$BASE/face_recognition/rknn"
51
52 sha256sum "$BASE/face_recognition/rknn/"*.rknn

```

量化 (fp)

```

ONNX checker: PASS
ONNX: /mnt/XAIU_Test/weights/w600k_mbf.onnx
输入: input.1 ['None', 3, 112, 112]
固定输入: [1, 3, 112, 112]
输出: 516 [1, 512]
预处理: 使用InsightFace常见外部归一化
RKNN mean=127.5, std=127.5

```

int_8

转换成功

RKNN: /mnt/XAIU_Test/face_recognition/rknn/w600k_mbf-int8.rknn

大小: 3.90 MiB

x00010000

```
D RKNN: [11:07:53.449] 47 ConvAdd 657 INT32 (256) | 0x00328d80 0x00329580 0x00000800
D RKNN: [11:07:53.449] 48 ConvPReLU 659 INT8 (512,256,1,1) | 0x00329580 0x00349580 0x00020000
D RKNN: [11:07:53.449] 48 ConvPReLU 660 INT32 (512) | 0x00349580 0x0034a580 0x00001000
D RKNN: [11:07:53.449] 48 ConvPReLU 696 INT16 (512,1,1) | 0x00355f80 0x00356380 0x00000400
D RKNN: [11:07:53.449] 49 ConvPReLU 662 INT8 (64,512,1,1) | 0x0034a580 0x00352580 0x00008000
D RKNN: [11:07:53.449] 49 ConvPReLU 663 INT32 (64) | 0x00352580 0x00352780 0x00000200
D RKNN: [11:07:53.449] 49 ConvPReLU 697 INT16 (64,1,1) | 0x00356380 0x00356400 0x00000080
D RKNN: [11:07:53.449] 50 Reshape 514_tp_rs_i1 INT64 (4) | 0x0037a400 0x0037a420 0x00000020
D RKNN: [11:07:53.449] 51 Conv fc.weight INT8 (512,3136,1,1) | 0x00000000 0x00188000 0x00188000
D RKNN: [11:07:53.449] 51 Conv fc.bias INT32 (512) | 0x00188000 0x00189000 0x00001000
D RKNN: [11:07:53.449] 52 Reshape 516-rs_i1 INT64 (2) | 0x0037a440 0x0037a450 0x00000010
D RKNN: [11:07:53.449] -----+-----
D RKNN: [11:07:53.449] -----
D RKNN: [11:07:53.449] Total Internal Memory Size: 820.75KB
D RKNN: [11:07:53.449] Total Weight Memory Size: 3563.12KB
D RKNN: [11:07:53.449] -----
D RKNN: [11:07:53.449] <<<<<<< end: rknn::RKNNMemStatisticsPass
```



 verify_w600k_fp_int8.py



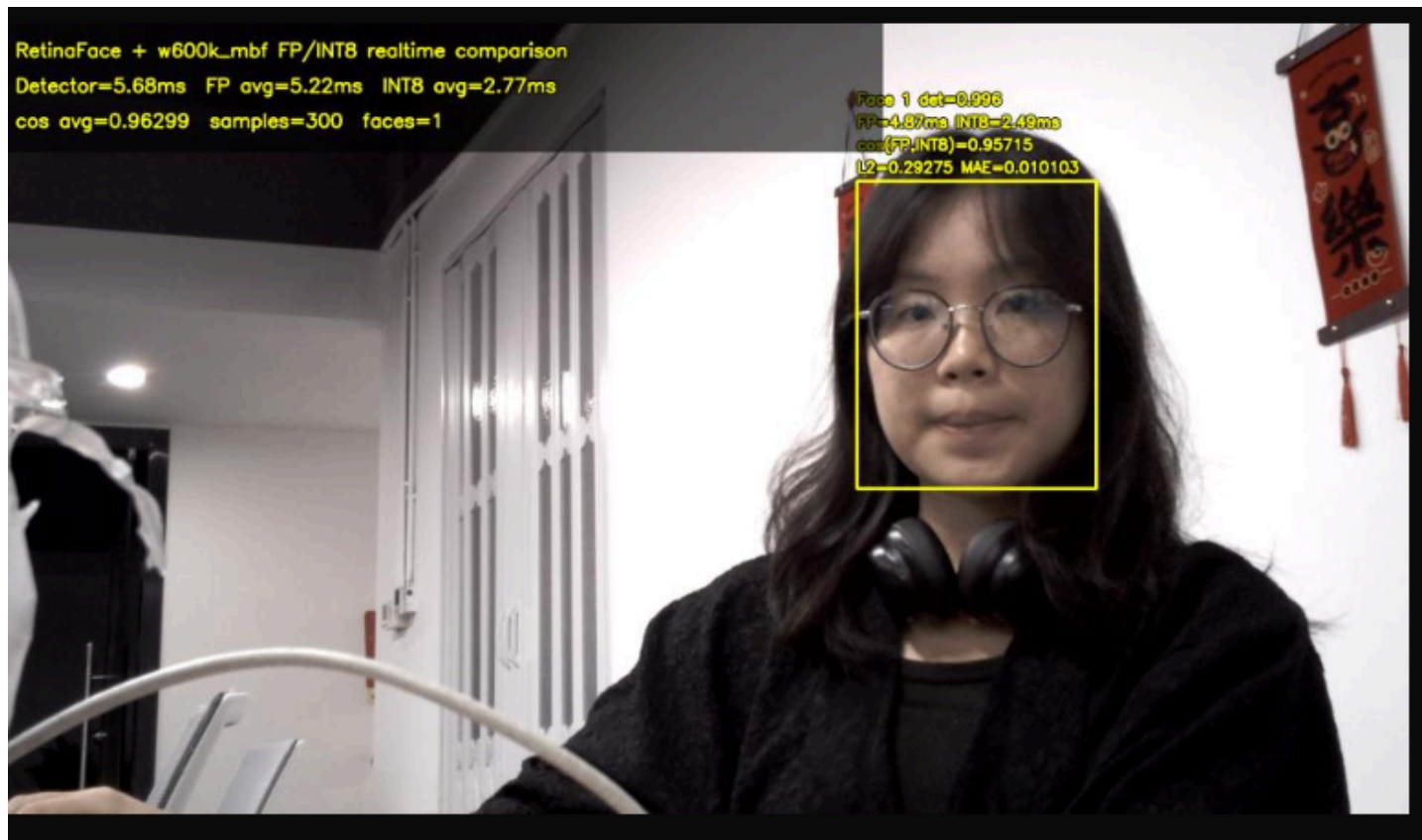
 realtime_w600k_fp_int8_compare.py

对比fp和int_8

不要拆行，整行复制：

代码块

```
1 source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/yolov8_rk3588/code
  && python realtime_w600k_fp_int8_compare.py --face-detector
    /root/yolov8_rk3588/code/model/RetinaFace_mobile320-int8.rknn --fp-model
    /root/yolov8_rk3588/code/model/w600k_mbf-fp.rknn --int8-model
    /root/yolov8_rk3588/code/model/w600k_mbf-int8.rknn --device /dev/video22 --
width 1920 --height 1080 --camera-fps 20 --compare-fps 5 --detector-core 0 --
fp-core 1 --int8-core 2 --detector-layout nhwc --recognizer-layout nhwc --
candidate-conf 0.02 --conf 0.30 --nms 0.50 --max-faces 2 --preview-width 960 --
jpeg-quality 75 --result-dir /root/yolov8_rk3588/result_w600k_compare --port
8082
```



RetinaFace检测置信度: 0.996

RetinaFace平均耗时: 5.68 ms

FP平均耗时: 5.22 ms

INT8平均耗时: 2.77 ms

FP/INT8平均余弦: 0.96299

当前帧FP/INT8余弦: 0.95715

L2误差: 0.29275

MAE: 0.010103

累计样本: 300

量化优化 (余弦值+量化图片500张)

```

P directory
● (myconda) root@ZDLAQA:~/Desktop/mnt/XAIU_Test# python prepare_face_quant_112.py \
--source /root/Desktop/mnt/XAIU_Test/lfw_quant_flat_500 \
--output /root/Desktop/mnt/XAIU_Test/face_recognition/calibration/lfw_112_500 \
--dataset /root/Desktop/mnt/XAIU_Test/face_recognition/calibration/dataset_500.txt \
--size 112 \
--overwrite

=====
源目录: /mnt/XAIU_Test/lfw_quant_flat_500
输出目录: /mnt/XAIU_Test/face_recognition/calibration/lfw_112_500
输入图片: 500
成功转换: 500
失败图片: 0
量化列表: /mnt/XAIU_Test/face_recognition/calibration/dataset_500.txt
=====
○ (myconda) root@ZDLAQA:~/Desktop/mnt/XAIU_Test# python convert_w600k_mbf_to_rknn.py \
--onnx /root/Desktop/mnt/XAIU_Test/weights/w600k_mbf.onnx \
--output /root/Desktop/mnt/XAIU_Test/weights/w600k_mbf-int8-calib500.rknn \
--mode int8 \
--dataset /root/Desktop/mnt/XAIU_Test/face_recognition/calibration/dataset_500.txt \
--target rk3588

=====
ONNX checker: PASS
ONNX: /mnt/XAIU_Test/weights/w600k_mbf.onnx
输入: input.1 ['None', 3, 112, 112]
固定输入: [1, 3, 112, 112]
输出: 516 [1, 512]
预处理: 使用InsightFace常见外部归一化
RKNN mean=127.5, std=127.5
=====

```

量化txt

```

python prepare_face_quant_112.py \ --source
/root/Desktop/mnt/XAIU_Test/lfw_quant_flat_500 \ --output
/root/Desktop/mnt/XAIU_Test/face_recognition/calibration/lfw_112_500 \ --dataset
/root/Desktop/mnt/XAIU_Test/face_recognition/calibration/dataset_500.txt \ --size 112 \ --
overwrite

```

量化rknn

```

python convert_w600k_mbf_to_rknn.py \ --onnx
/root/Desktop/mnt/XAIU_Test/weights/w600k_mbf.onnx \ --output
/root/Desktop/mnt/XAIU_Test/weights/w600k_mbf-int8-calib500.rknn \ --mode int8 \ --
dataset /root/Desktop/mnt/XAIU_Test/face_recognition/calibration/dataset_500.txt \ --target
rk3588

```

实时检测

```

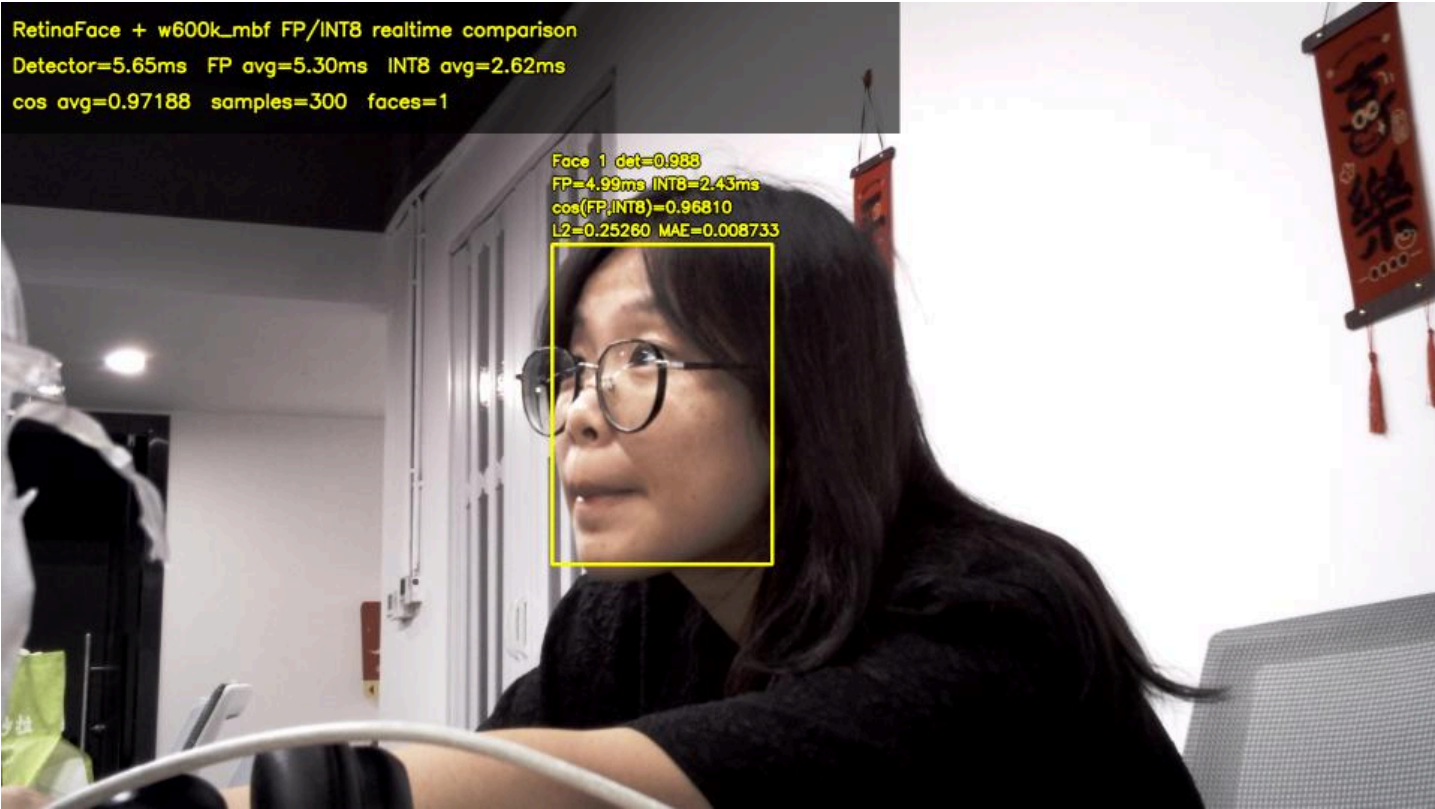
source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/yolov8_rk3588/code && python
realtime_w600k_fp_int8_compare.py --face-detector
/root/yolov8_rk3588/code/model/RetinaFace_mobile320-int8.rknn --fp-model

```

```
/root/yolov8_rk3588/code/model/w600k_mbf-fp.rknn --int8-model
/root/yolov8_rk3588/code/model/w600k_mbf-int8-calib500.rknn --device /dev/video22 --width
1920 --height 1080 --camera-fps 20 --compare-fps 5 --detector-core 0 --fp-core 1 --int8-core 2 --
detector-layout nhwc --recognizer-layout nhwc --candidate-conf 0.02 --conf 0.30 --nms 0.50 --
max-faces 2 --preview-width 960 --jpeg-quality 75 --result-dir
/root/yolov8_rk3588/result_w600k_compare_calib500 --port 8082
```

对比结果

指标	旧100张量化	新500张量化	变化
FP/INT8平均余弦	0.96299	0.973082	0.010092
FP平均耗时	5.22 ms	5.275 ms	基本不变
INT8平均耗时	2.77 ms	2.637 ms	略有提升
INT8相对FP速度	约1.88倍	约2.00倍	改善



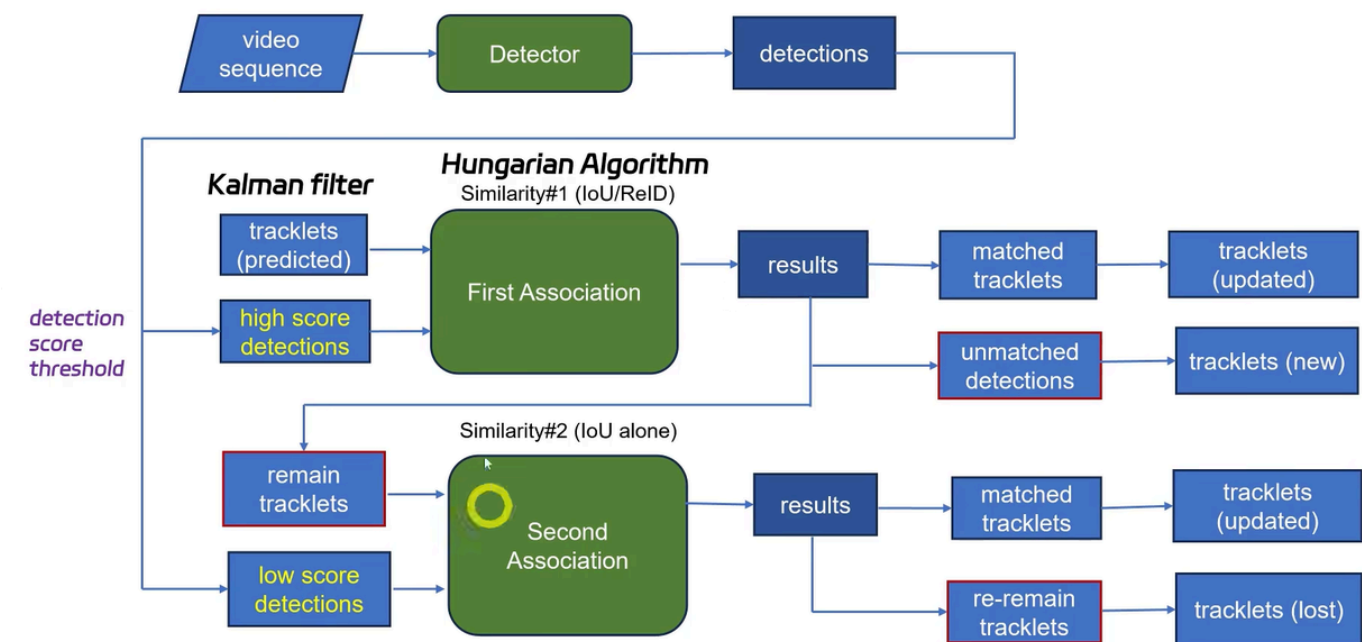
```
{(rknn310) root@RK3588-Tronlong:~/yolov8_rk3588/code# python -c "import json; p='/root/yolov8_rk3588/result_w600k_compar
e_calib500/latest_status.json'; d=json.load(open(p)); print('samples=',d.get('sample_count')); print('cosine_avg=',d.get
('cosine_avg')); print('fp_avg_ms=',d.get('fp_avg_ms')); print('int8_avg_ms=',d.get('int8_avg_ms'))"
samples= 300
cosine_avg= 0.973082
fp_avg_ms= 5.275
int8_avg_ms= 2.637
(rknn310) root@RK3588-Tronlong:~/yolov8_rk3588/code# |
```

ByteTrack算法 (id追踪)

论文: <https://arxiv.org/pdf/2110.06864>

算法原理: <https://www.bilibili.com/video/BV1Mu411A7dm?t=0.3>

ByteTrack算法流程框图



ByteTrack多目标跟踪性能

Similarity#1	Similarity#2	MOT17			BDD100K		
		MOTA↑	IDF1↑	IDs↓	mMOTA↑	mIDF1↑	IDs↓
IoU	Re-ID	75.8	77.5	231	39.2	48.3	29172
IoU	IoU	76.6	79.3	159	39.4	48.9	27902
Re-ID	Re-ID	75.2	78.7	276	45.0	53.4	10425
Re-ID	IoU	76.3	80.5	216	45.5	54.8	9140

Table 1. Comparison of different type of similarity metrics used in the first association and the second association of BYTE on MOT17 and BDD100K validation set. The best results are shown in **bold**.

Method	w/ Re-ID	MOT17			BDD100K			FPS
		MOTA↑	IDF1↑	IDs↓	mMOTA↑	mIDF1↑	IDs↓	
SORT		74.6	76.9	291	30.9	41.3	10067	30.1
DeepSORT	✓	75.4	77.2	239	24.5	38.2	10720	13.5
MOTDT	✓	75.8	77.6	273	26.7	39.8	14520	11.1
BYTE (ours)		76.6	79.3	159	39.4	48.9	27902	29.6
BYTE (ours)	✓	76.3	80.5	216	45.5	54.8	9140	11.8

Table 2. Comparison of different data association methods on MOT17 and BDD100K validation set. The best results are shown in **bold**.

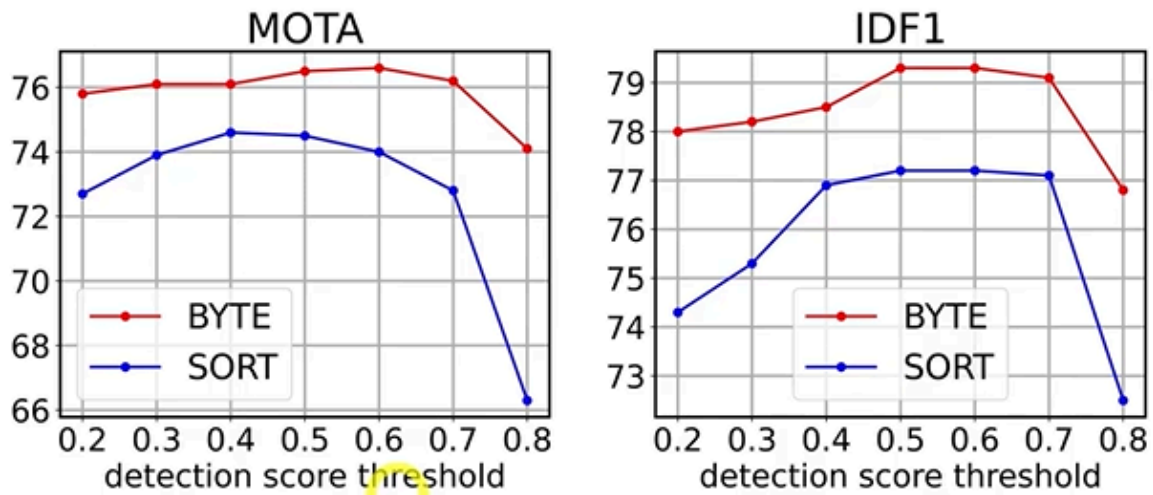


Figure 3. Comparison of the performances of BYTE and SORT under different detection score thresholds. The results are from the validation set of MOT17.

工作台空岗算法

参考资料：

https://github.com/deepinsight/insightface/tree/master/model_zoo

https://github.com/deepinsight/insightface/releases?utm_source=chatgpt.com

项目拆解

收银台空岗检测：业务判定公式与数据流

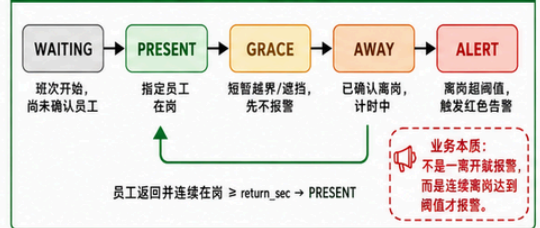
适用：single / per_employee / minimum_staff

从摄像头一帧数据，到在岗/离岗/告警结果的完整业务链路

A. 数据流总览（横向流程图）



D. 状态机时间轴示意图



C. 业务判定核心公式（公式框，重点突出）

1 脚点提取（推荐）

$$p_{foot} = \frac{ankle_l + ankle_r}{2}$$

若单脚可见： $p_{foot} = visible_ankle$
若脚跟不可见： $p_{foot} = bbox_bottom_center$

💡 优先脚踵(15, 16)，因为最接近地面，最适合判定定位。

2 ROI 在岗判定

$$d = pointPolygonTest(ROI, p_{foot}, measureDist=True)$$
$$inside = 1, \text{ if } d \geq -\epsilon$$
$$inside = 0, \text{ if } d < -\epsilon$$

💡 ϵ 为边界容差，常取 3~10 px，用来减少边界抖动。

3 身份绑定判定

$$bind(track_i, emp_j) = 1$$

if $sim(face_i, emp_j) \geq \tau_{id}$
and $overlap(face_box_i, body_box_i) \geq \tau_{match}$
TTL = 20 s

💡 人脸识别到身份后，绑定到 Track ID；短时遮挡时身份可保持。

4 员工在岗判定

$$present(emp_k, t) = bound(emp_k, track_i) \times inside(track_i, ROI, t)$$

💡 已绑定 + 在 ROI 内，才算该员工在岗。

5 三种排班模式

模式	判定公式	说明
single	$present = 1$ (指定员工在 ROI 内)	一人一岗
per_employee	$state(emp_k)$ 独立运行	多人共两岗分别告警
minimum_staff	$present_count = \sum present(emp_k)$	统计总在岗人数

$shortage = 1, \text{ if } present_count < N_{min}$
 $shortage = 0, \text{ if } present_count \geq N_{min}$

6 状态机与告警时间

若 $inside=1$ 持续 ≥ return_sec → PRESENT
若 $inside=0$ 且 $away_duration < grace_sec$ → GRACE
若 $inside=0$ 且 $grace_sec \leq away_duration < alert_sec$ → AWAY
若 $inside=0$ 且 $away_duration \geq alert_sec$ → ALERT

默认生产参数：grace=5s, alert=180s, return=2s

🚨 告警条件 = 连续离岗时间达到 alert_sec

E. 业务例子：E001 收银员离岗事件



带走的
5 句话

1 先看脚点，再看 ROI。

2 人脸负责知道是谁，跟踪负责保持连续性。

3 FrameSlot 丢旧帧，换低延迟。

4 minimum_staff 模式看人数，不看某一个人。

5 真正触发告警的是连续离岗时间，而不是瞬时越界。

模块	一句话作用	输出
YOLOv8-Pose	找到人体并标出身体关节点	人体框 + 17个人体关键点
ByteTrack	判断前后帧是不是同一个人	Track ID
RetinaFace	找到人脸并标出人脸5点	人脸框 + 5个人脸关键点
MobileFaceNet	判断这张脸是谁	512维人脸特征 embedding

涉及知识点

第一阶段：空岗核心算法

- YOLOv8-Pose;
- Ultralytics Track;
- ByteTrack;
- pointPolygonTest;
- 人脸检测、对齐和识别

第二阶段：实时系统问题

6. threading;
7. queue;
8. FastAPI WebSocket;
9. Background Tasks;
10. V4L2

`CameraService` 不是某个固定AI模型，而是你项目中设计的一个摄像头服务模块。

它的职责通常包括：

1. 打开 `/dev/video22`
2. 设置分辨率、像素格式、帧率
3. 启动独立采集线程
4. 持续读取最新图像
5. 给图像添加时间戳和帧序号
6. 把最新帧发布给 `Pose`、人脸识别、Web预览
7. 检测断流并尝试恢复
8. 程序退出时释放摄像头

第三阶段：模型部署问题

11. ONNX结构;
12. ONNX Checker;
13. RKNN-Toolkit2;
14. RKNN Model Zoo和YOLO后处理

第四阶段：数据和交付问题

15. SQLite外键;
16. 数据库事务;
17. Precision、Recall和F1;
18. Git;
19. Git LFS

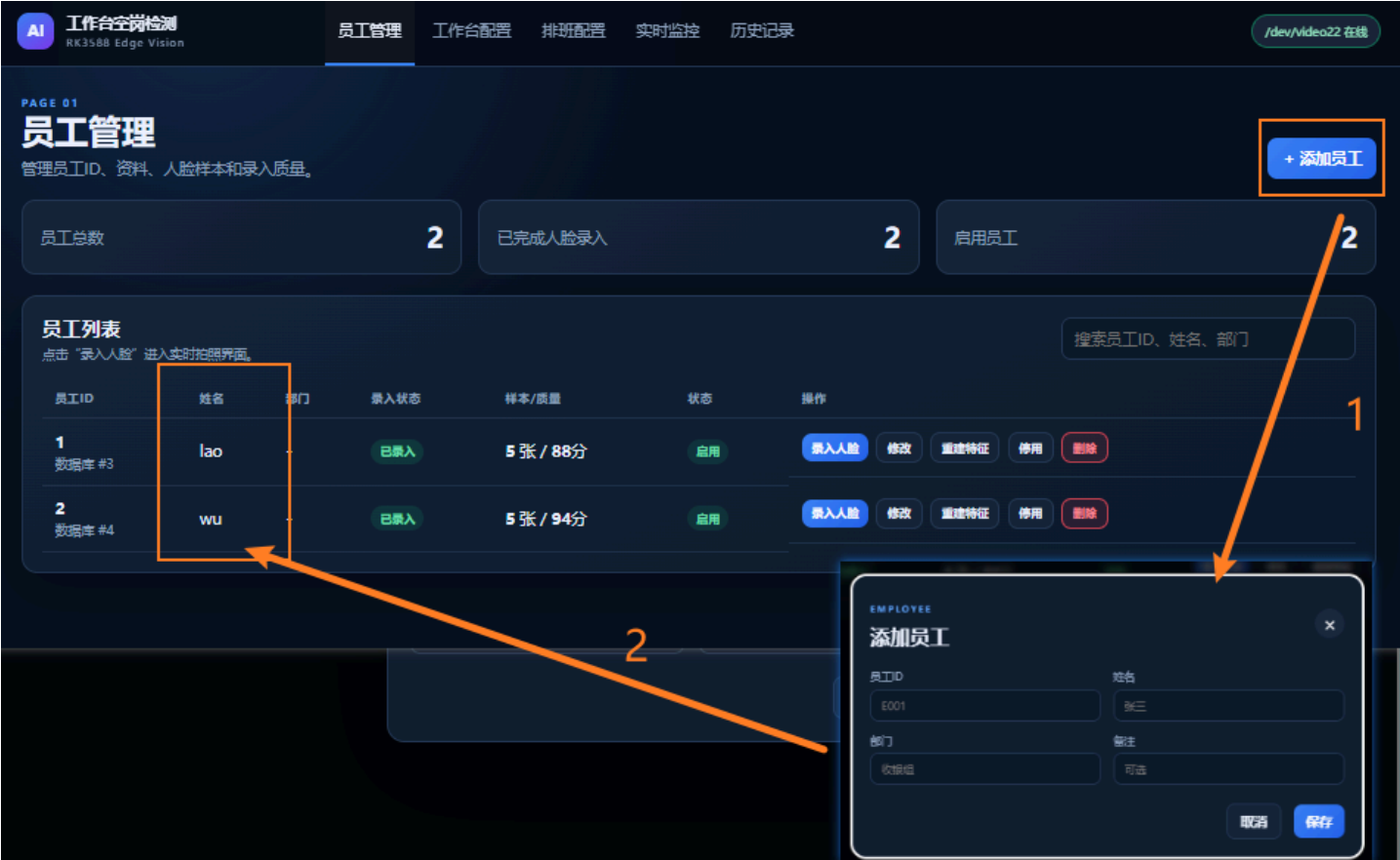
网页UI设计

页面1：员工管理

功能：

代码块

- 1 添加员工
- 2 修改员工
- 3 分配员工ID
- 4 上传5~10张照片（拍照界面，手动定
- 5 1. 正脸
- 6 2. 轻微向左
- 7 3. 轻微向右
- 8 4. 轻微抬头或低头
- 9 5. 正常工作姿态
- 10
- 11 UI可以显示（实际）：
- 12 正脸 已完成
- 13 轻微左转 已完成
- 14 轻微右转 待拍摄
- 15 正常姿态 待拍摄
- 16 补充照片 待拍摄
- 17)
- 18 重新生成人脸特征
- 19 停用员工
- 20 查看录入质量
- 21



检查项目	当前要求
人脸数量	必须正好1张
人脸宽度	≥ 100 px
清晰度	≥ 45
RetinaFace置信度	≥ 0.60
平均亮度	45~220
左右头部倾斜	绝对值 $\leq 25^\circ$
摄像头帧年龄	≤ 2 秒

尺寸分 = $\min(100, \text{人脸宽度} / 100 \times 70)$

清晰度分 = $\min(100, \text{清晰度} / 45 \times 70)$

亮度分 = $\max(0, 100 - |\text{亮度} - 132| \times 0.8)$

置信度分 = $\text{RetinaFace置信度} \times 100$

页面2：工作台配置

功能：

代码块

1 选择摄像头

- 2 实时预览
- 3 Canvas手动画ROI
- 4 设置离岗阈值
- 5 保存ROI
- 6 测试区域判断
- 7
- 8
- 9 优化
- 10 single
- 11 一人一岗，当前逻辑
- 12
- 13 per_employee
- 14 同一ROI可排多人，每个人独立离岗告警
- 15
- 16 minimum_staff
- 17 同一ROI可排多人，只监控最低在岗人数

PAGE 02

工作台配置

先选择工作台，再在画面上绘制ROI。测试按钮直接使用当前画布草稿。

摄像头与ROI

当前编辑: HHH (ID 1), 红色多边形为当前ROI草稿。

当前摄像头

/dev/video22 - rkisp_mainpath (platformrkisp0-vir0)

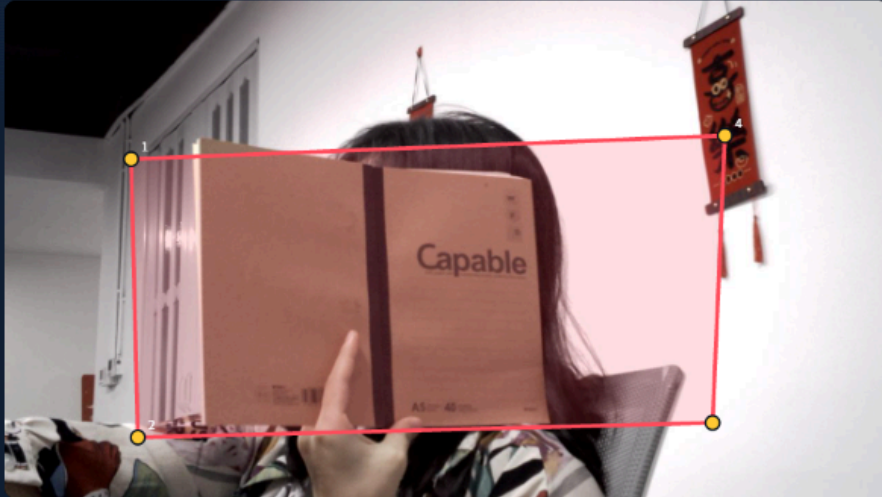
撤销一点

清空画布草稿

清空已保存ROI

保存ROI

测试当前ROI



ROI点数: 4, 当前草稿可测试或保存。

尚未测试区域。

已配置工作台

点击卡片即可选中并加载其ROI。

HHH

/dev/video22

ROI点数

4

告警阈值

20秒

状态

启用

参数自定义

+ 新建工作台

参数

新建工作台后会自动选中; 也可以点击下方工作台卡片进行编辑。

工作台

HHH

名称

HHH

绑定摄像头

/dev/video22

离开告警阈值 (秒)

20

短暂丢失容忍 (秒)

5

返回稳定确认 (秒)

2

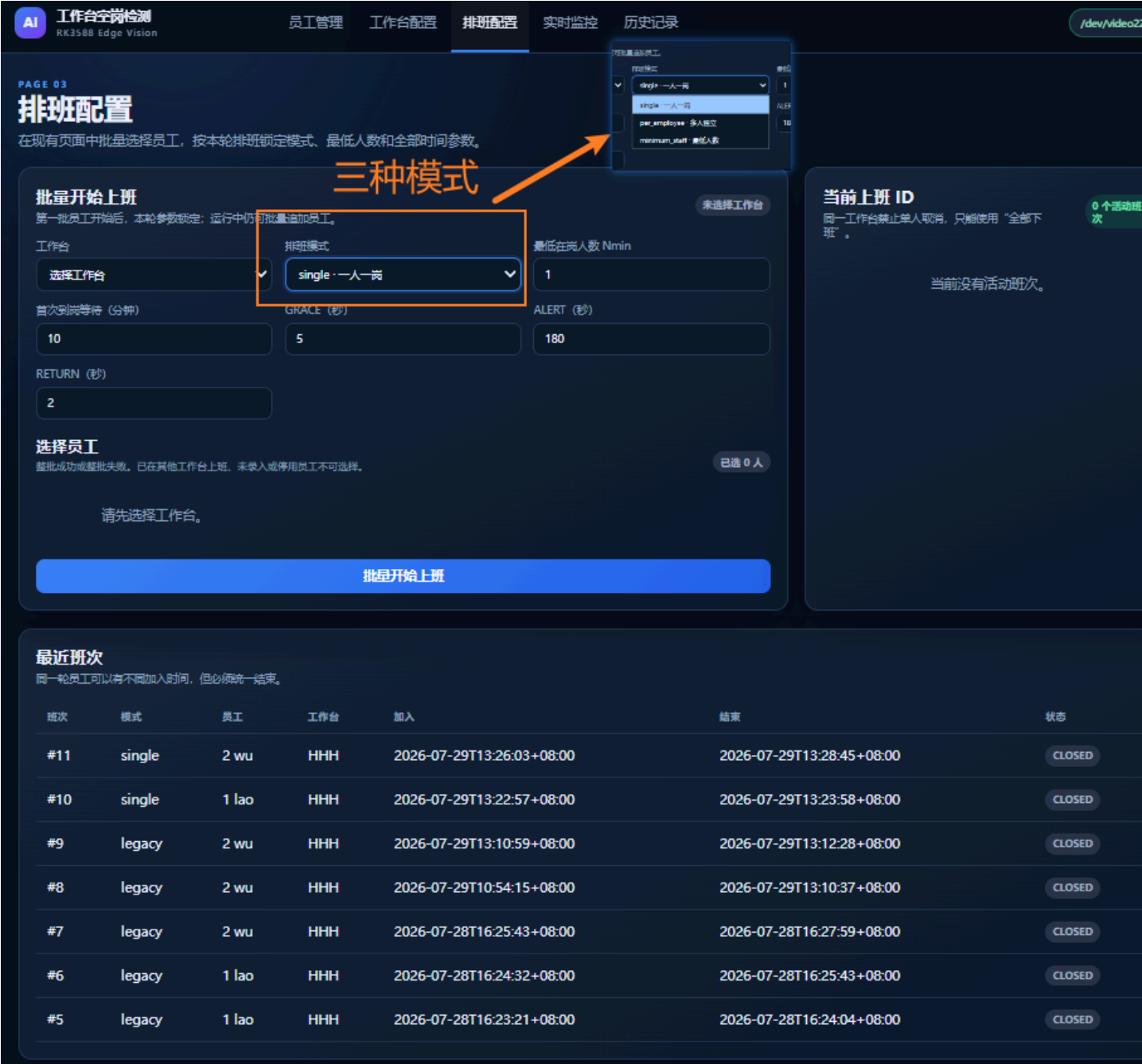
☒ 启用工作台

保存工作台参数

删除当前工作台

告警生效条件

工作台必须保存至少3个ROI点; 排班员工必须先被人脸识别, 并进入ROI确认“在岗”。之后离开ROI才开始倒计时和写入历史记录。



页面3：排班配置

功能：

代码块

1

选择员工

2

选择工作台

3

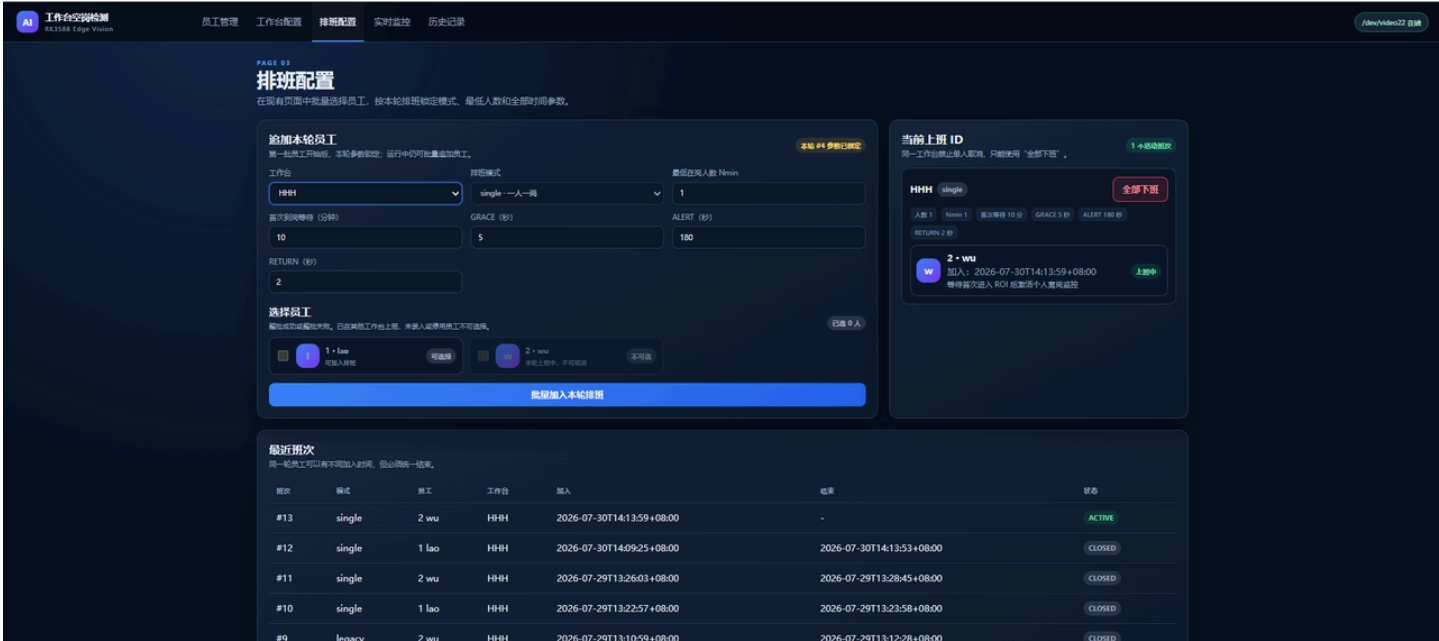
点击“开始上班”

4

点击“结束上班”

5

显示当前上班ID



页面4：实时监控

状态颜色：

代码块

- 1 绿色：在岗
- 2 黄色：疑似离岗/倒计时
- 3 红色：空岗告警
- 4

显示：

代码块

- 1 员工姓名
- 2 员工ID
- 3 工作台
- 4 当前状态
- 5 离岗持续时间
- 6 告警倒计时
- 7 最新识别时间
- 8 实时视频

视觉元素	含义
黄色/绿色大框	Pose/Track的人体框，旁边通常带Track ID
紫色小点和连线	置信度达到阈值的17个关键点和骨架

红色人脸框	RetinaFace检测到的人脸区域
ROI多边形线	工作台区域，业务规则判断锚点是否在内
UNKNOWN	当前Track尚未匹配到员工身份

AI

工作台空岗检测

RK3588 Edge Vision

员工管理工作台配置排班配置实时监控历史记录

/dev/Video22 在线

PAGE 04

实时监控

人脸检测红框 绿色是人脸识别 黄框工作台范围

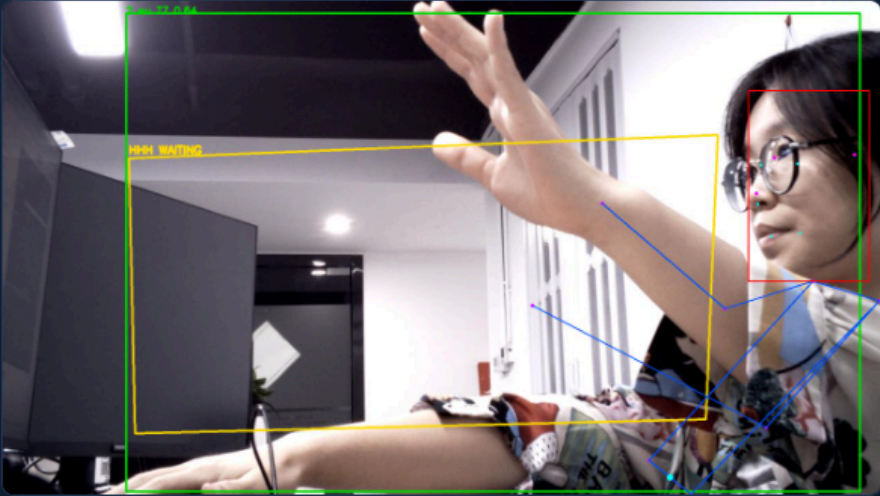
打开当前截图

绿色在岗、黄色疑似离岗或倒计时、红色空岗告警。

实时监控

YOLOv8-Pose + ByteTrack + RetinaFace + W600K

LIVE



Pose 24.05 msRetinaFace 5.56 msW600K 2.55 ms

3个模型

工作台状态

区域缺员与个人缺量同时监控。

HHH • single

等待首次识别

在岗 0/1 · 等待首次到岗 1 · 区域缺员 0s · 暂停额度剩余 230s
存在首次到岗等待员工，区域缺员计时已暂停

1 • lao

等待首次识别

首次到岗剩余230s
缺勤持续时间0s
告警倒计时180s
最新识别时间-
相似度-
Track ID-

HHH

等待员工首次识别并进入 ROI

当前识别人员

显示Track ID、员工ID、相似度和最新识别时间。

TRACK	员工	相似度	最新识别	人体置信度
T7	2 wu	0.6136	2026-07-30T14:10:11+08:00	0.853

PAGE 04

实时监控

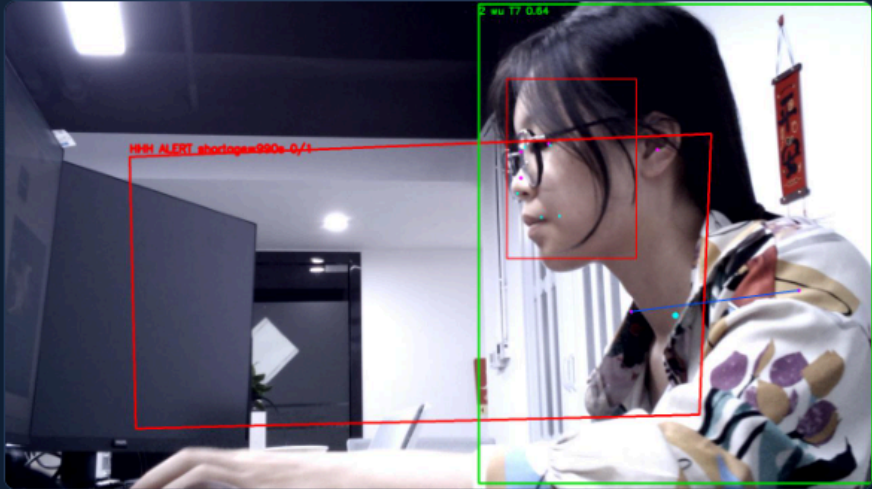
绿色在岗、黄色疑似离岗或倒计时、红色空岗告警。

打开当前截图

实时视频

YOLOv8-Pose + ByteTrack + RetinaFace + W600K

LIVE



Pose 25.7 ms

RetinaFace 5.58 ms

W600K 0 ms

工作台状态

区域缺员与个人缺勤同时监控。

HHH • single

空岗告警

在岗 0/1 - 等待首次到岗 0 - 区域缺员 981s - 暂停额度剩余 0s
区域缺员告警

1 • lao

HHH

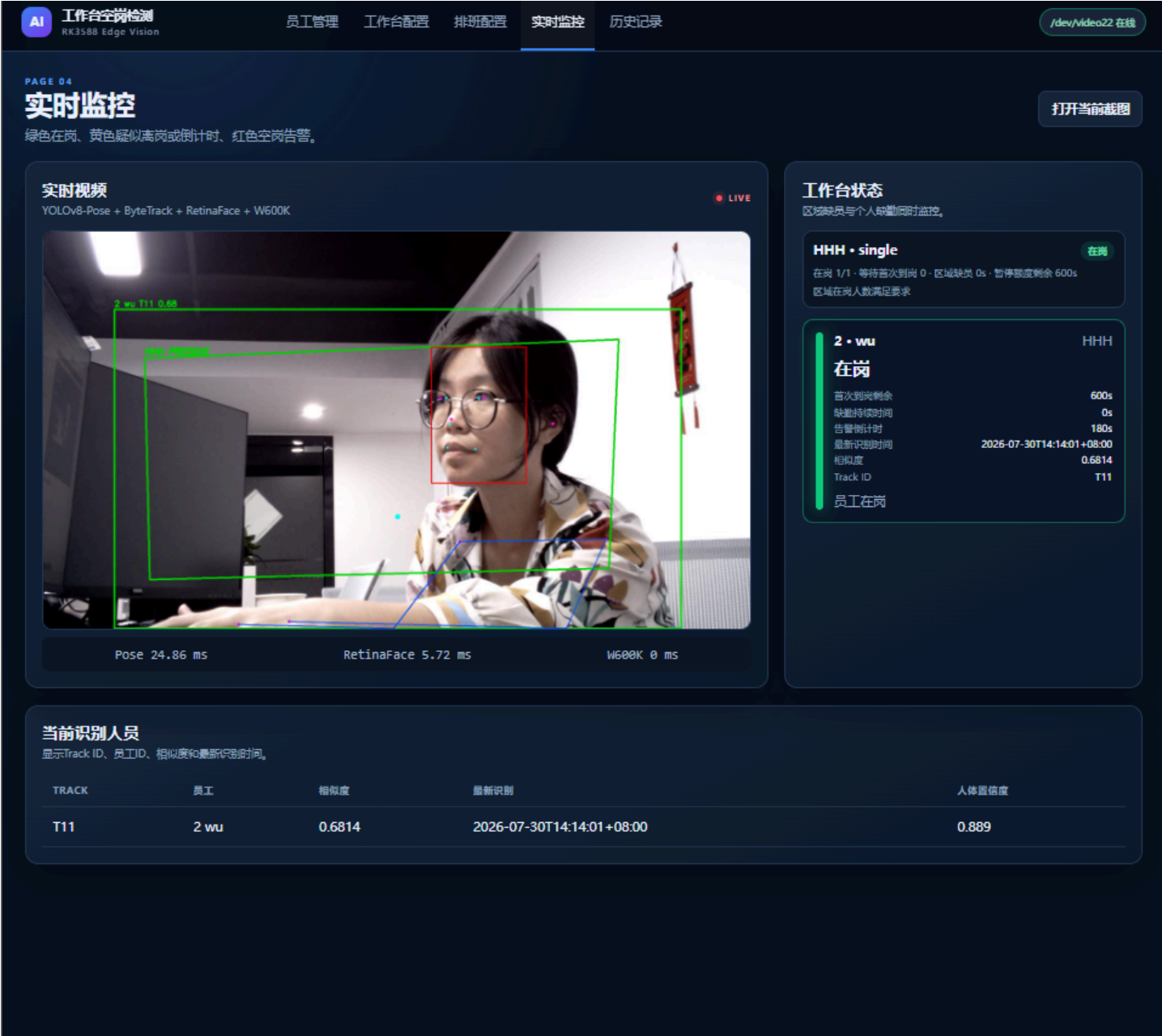
未到岗/迟到

首次到岗剩余 0s
缺勤持续时间 0s
告警倒计时 180s
最新识别时间 -
相似度 -
Track ID -
超过首次到岗等待上限，已记录未到岗/迟到

当前识别人员

显示Track ID、员工ID、相似度和最新识别时间。

TRACK	员工	相似度	最新识别	人体置信度
T7	2 wu	0.6359	2026-07-30T14:10:27+08:00	0.808



页面5：历史记录

功能：

代码块

- 1 按员工查询
- 2 按工作台查询
- 3 按日期查询
- 4 查看离开时间
- 5 查看回来时间
- 6 查看离岗时长
- 7 查看告警截图
- 8 导出CSV

工作智能终端RK3588 Edge System

员工管理工作台配置排班配置实时监控历史记录

历史记录

历史记录

个人排班、区域缺员和请假审批统一在一张表中查询，请前往配置项下配置申请。

选择排班

员工

工作台

开始日期

结束日期

查询

导出 CSV

删除当前排班

考勤与区域事件

共有 8 条记录，如需要筛选，请个人排班以“已批准请假” 页码进行筛选。

排班	员工 / 区域	工作台	关联事件	开始	结束	时长	状态	备注	操作
已批准请假	2 vu	1991	fb3d40b3	2026-07-30T14:40:05+08:00	-	2026-07-30T14:40:11+08:00	28 秒	开始	已批准请假
已批准请假	2 vu	1991	fb3d40b3	2026-07-30T14:40:05+08:00	-	2026-07-30T14:40:11+08:00	32 秒	开始	已批准请假
已批准请假	2 vu	1991	69e821d8	2026-07-30T14:38:52+08:00	2026-07-30T14:39:19+08:00	2026-07-30T14:39:43+08:00	316 秒	开始	已批准请假
已批准请假	2 vu	1991	69e821d8	2026-07-30T14:38:52+08:00	2026-07-30T14:39:19+08:00	2026-07-30T14:39:42+08:00	315 秒	开始	已批准请假
已批准请假	2 vu	1991	a2be349d	2026-07-30T14:37:54+08:00	2026-07-30T14:38:24+08:00	2026-07-30T14:38:27+08:00	192 秒	开始	已批准请假
已批准请假	2 vu	1991	a2be349d	2026-07-30T14:37:54+08:00	2026-07-30T14:38:24+08:00	2026-07-30T14:38:27+08:00	191 秒	开始	已批准请假
已批准请假	2 vu	1991	-	2026-07-29T13:12:05+08:00	2026-07-29T13:12:25+08:00	-	- 秒	开始	已批准请假
已批准请假	2 vu	1991	-	2026-07-29T13:15:27+08:00	2026-07-29T13:15:47+08:00	-	- 秒	开始	已批准请假

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	事件ID	记录分类	事件类型	关联事件ID	员工ID	员工姓名	工作台	离开/未到	告警时间	返回/结束	持续时长	事件状态	请假状态	请假备注	离开截图	告警截图	返回截图									
2	29	LEAVE_AFF	个人缺勤	fb3d40b3c	2 vu	HHH		2026-07-30T14:40:05+08:00	2026-07-30T14:40:11+08:00	28	CLOSED	APPROVED	111	/root/workstation_absence/data/events/2026-07-30/shift_14_144007_404306_leave.jpg												
3	28	REGION	区域缺员	fb3d40b3c	2 vu	HHH		2026-07-30T14:40:05+08:00	2026-07-30T14:40:11+08:00	32	CLOSED	NONE		/root/workstation_absence/data/events/2026-07-30/round_5_144006_838034_region_leave.jpg												
4	27	REGION	区域缺员	69e821d8c	2 vu	HHH		2026-07-30T14:38:52+08:00	2026-07-30T14:39:19+08:00	316	CLOSED	NONE		/root/woz/root/woz/root/workstation_absence/data/events/2026-07-30/round_5_143943_082353_region_return.jpg												
5	26	ABSENCE	个人缺勤	69e821d8c	2 vu	HHH		2026-07-30T14:38:52+08:00	2026-07-30T14:39:19+08:00	315	CLOSED	NONE		/root/woz/root/woz/root/workstation_absence/data/events/2026-07-30/round_5_143942_728403_return.jpg												
6	25	REGION	区域缺员	a2be349dc	2 vu	HHH		2026-07-30T14:37:54+08:00	2026-07-30T14:38:24+08:00	192	CLOSED	NONE		/root/woz/root/woz/root/workstation_absence/data/events/2026-07-30/round_5_143827_501893_region_return.jpg												
7	24	ABSENCE	个人缺勤	a2be349dc	2 vu	HHH		2026-07-30T14:37:54+08:00	2026-07-30T14:38:24+08:00	191	CLOSED	NONE		/root/woz/root/woz/root/workstation_absence/data/events/2026-07-30/shift_14_143827_087876_return.jpg												
8	16	ABSENCE	个人缺勤		2 vu	HHH		2026-07-29T13:12:05+08:00	2026-07-29T13:12:25+08:00		OPEN	NONE		/root/woz/root/workstation_absence/data/events/2026-07-29/shift_9_131225_040517_return.jpg												
9	15	ABSENCE	个人缺勤		2 vu	HHH		2026-07-29T13:15:27+08:00	2026-07-29T13:15:47+08:00		OPEN	NONE		/root/woz/root/workstation_absence/data/events/2026-07-29/shift_8_111547_070129_return.jpg												
10																										
11																										

页面6：npu量化数据展示

工作智能终端

RK3588 Edge System

员工管理

工作台配置

排班配置

实时监控

历史记录

NPU量化

设备控制

rknnlite2.3.2

PAGE 06

NPU量化与推理数据

数据直接来自当前进程中的 RKNNLite inference() 调用；延迟为板级真实推理耗时，不包含图像预处理与后处理。

打开JSON

实时监控

NPU状态

真实NPU推理中

RK3588 - RKNNLite

模型/Runtime

2.3.2 / 2.0.0b0

Toolkit / Worknet

NPU延迟

0.96

频率 950 MHz

板级延迟

8.77 FPS

运行 00:01:23

实时板级量化与NPU指标

"NPU已验证" 必需同时满足 init_runtime 成功且 inference() 已至少执行一次。

LIVE

模型	设备/核心	量化	模型大小	输入	推理次数	实际帧率FPS	平均/PSA/PPSS	置信FPS	NPU验证	
YOLOv8-Pose	/root/yolov8_RK3588/code/model/yolov8n-pose-int8.ronn	RK3588 NPU Core 0	INT8 Toolkit 2.3.2	4.977 MB	1×640×640×3·uint8	727 帧/秒 0	8.79	25.50 / 24.94 / 26.69 ms	39.22	NPU已验证
RetinaFace	/root/yolov8_RK3588/code/model/RetinaFace_mobilenet-1.0-int8.ronn	RK3588 NPU Core 1	INT8 Toolkit 2.3.2	17.979 MB	1×320×320×3·uint8	220 帧/秒 0	2.65	6.03 / 5.76 / 7.34 ms	165.95	NPU已验证
W600K-MBF	/root/yolov8_RK3588/code/model/w600k_mbf-int8-calib500.ronn	RK3588 NPU Core 2	INT8 Toolkit 2.3.2	3.905 MB	1×112×112×3·uint8	48 帧/秒 0	0.58	2.76 / 2.60 / 3.21 ms	362.32	NPU已验证

运行管理

区分 NPU 模型推理速度，实际推理模型和人员置信结果。

推理FPS

8.77 FPS

Pose推理速度

8.67 FPS

人脸识别速度

2.65 FPS

检测延迟

8.67 FPS

当前人数

1

置信率

0

NPU load: Core0: 10%, Core1: 1%, Core2: 0%

页面7：手机端数据

运行状态与远程控制

无需登录，页面每 3 秒自动刷新。检测服务停止后，8091 控制后台继续在线。

2026-07-30 17:05:59

设备状态

在线

控制后台正常

真实IP地址

192.168.10.66

当前首选全局 IPv4

检测服务

运行中

sysv

运行时间

01:01:30

RK3588 系统 Uptime

CPU温度

36.1°C

最高 thermal zone

内存使用

748.5 MB / 15.6 GB

4.7%

磁盘使用

20.8%

2.9 GB / 13.7 GB

摄像头

在线 • /dev/video22

设备节点存在

活动班次

0

当前 ACTIVE 排班

开放告警

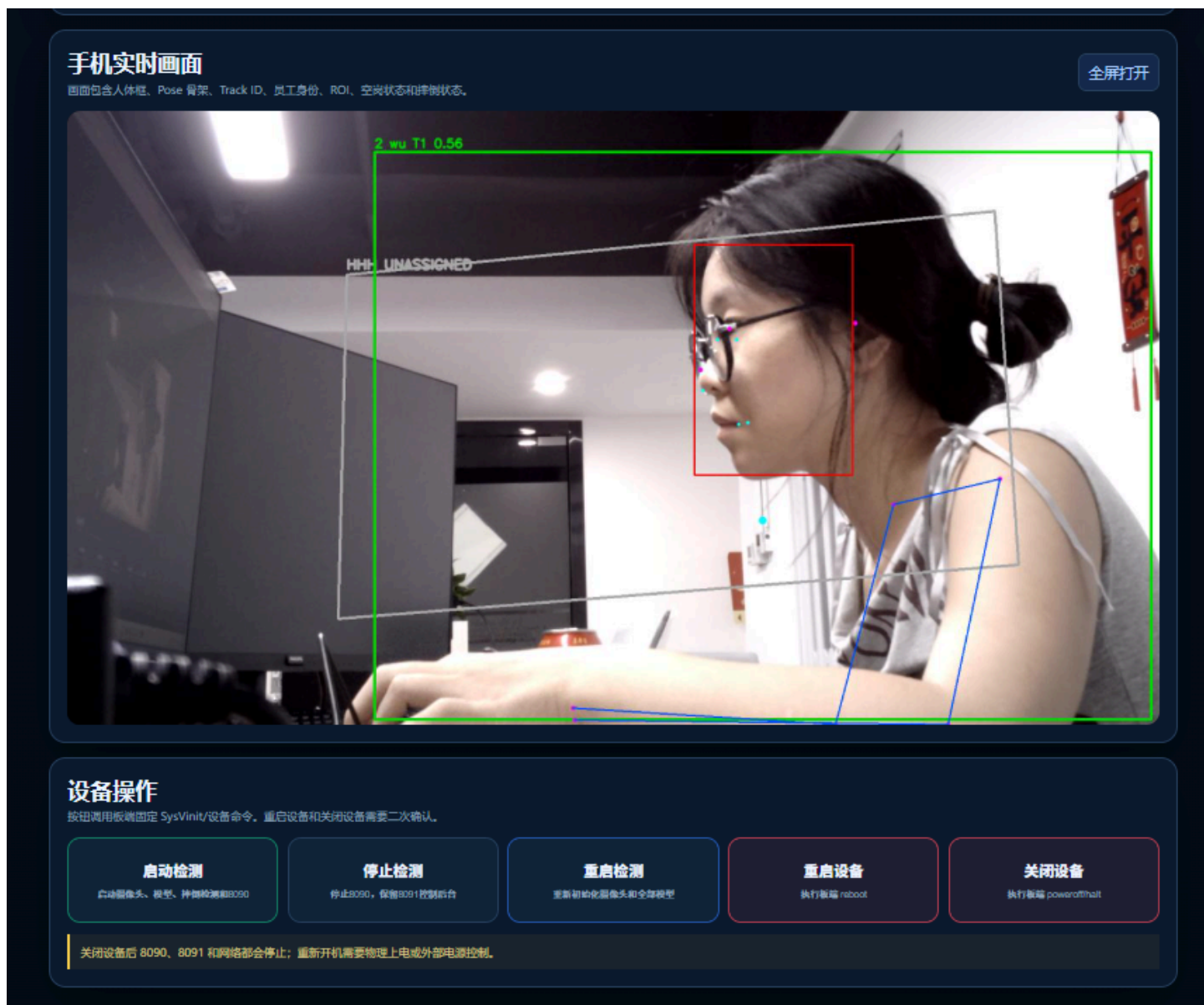
2

其中摔倒 0

模型、摔倒检测与真实 IP

摔倒检测作用于摄像头全画面内每一个 Pose Track，包括未绑定员工身份的 UNKNOWN 人员。

Pose模型	正常	有线网络 eth1 • 192.168.10.66	实时监控 后台数据
人脸检测模型	正常		
人脸识别模型	正常	网络接口 docker0 • 172.17.0.1	实时监控 后台数据
摔倒检测	正常		



整合项目

worker16



V1.0.6

单摄基础业务

worker17



V1.0.7.5

单摄 + 技术诊断 + 性能量化

worker18



V1.8.0

Camera A /dev/video22

Camera B /dev/video31

双区域独立检测

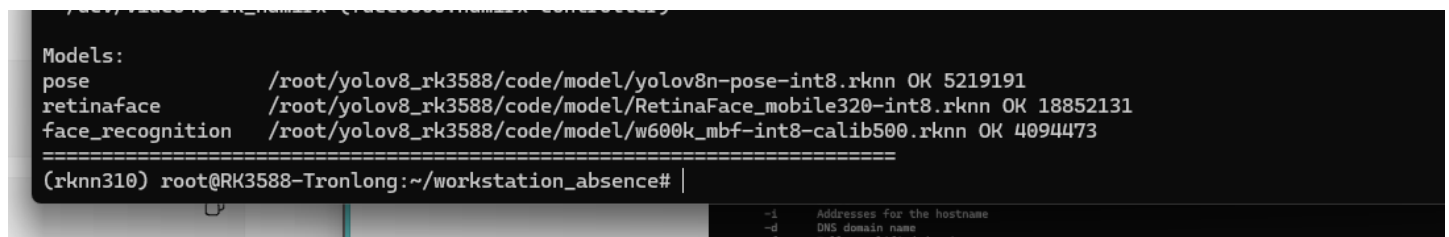
注意

```
/root/workstation_absence/data/workstation.db
```

因为这个文件还包含员工、工作台、排班轮次、班次和配置数据

```
source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && export  
PYTHONPATH=/root/workstation_absence && python tools/check_environment.py --config  
config.json
```

```
Models:  
pose          /root/yolov8_rk3588/code/model/yolov8n-pose-int8.rknn OK 5219191  
retinaface    /root/yolov8_rk3588/code/model/RetinaFace_mobile320-int8.rknn OK 18852131  
face_recognition /root/yolov8_rk3588/code/model/w600k_mbf-int8-calib500.rknn OK 4094473  
=====
```



(rknn310) root@RK3588-Tronlong:~/workstation_absence# |

检查网页依赖

```
source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && python -  
c "import fastapi,uvicorn,jinja2,multipart; print('Web dependencies OK')"
```

语法检查

```
source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && export  
PYTHONPATH=/root/workstation_absence && python -m py_compile workstation/runtime.py  
workstation/identity.py workstation/presence.py workstation/web.py
```

重新启动:

代码块

```
1 source /root/yolov8_rk3588/rknn310/bin/activate && cd  
/root/workstation_absence && export PYTHONPATH=/root/workstation_absence &&  
python app.py --config config.json
```

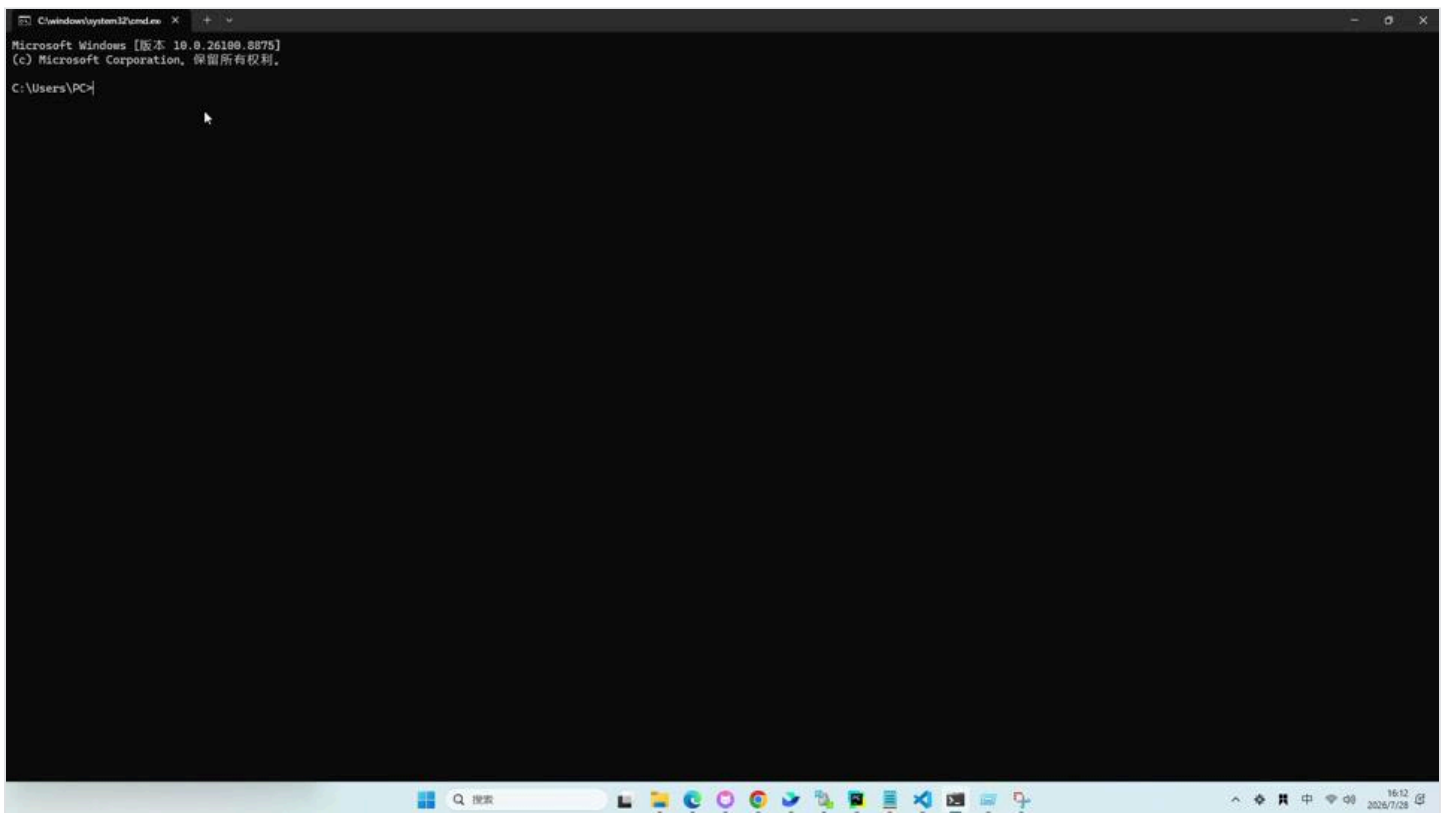
1.6v

输入worker16

它等价于：

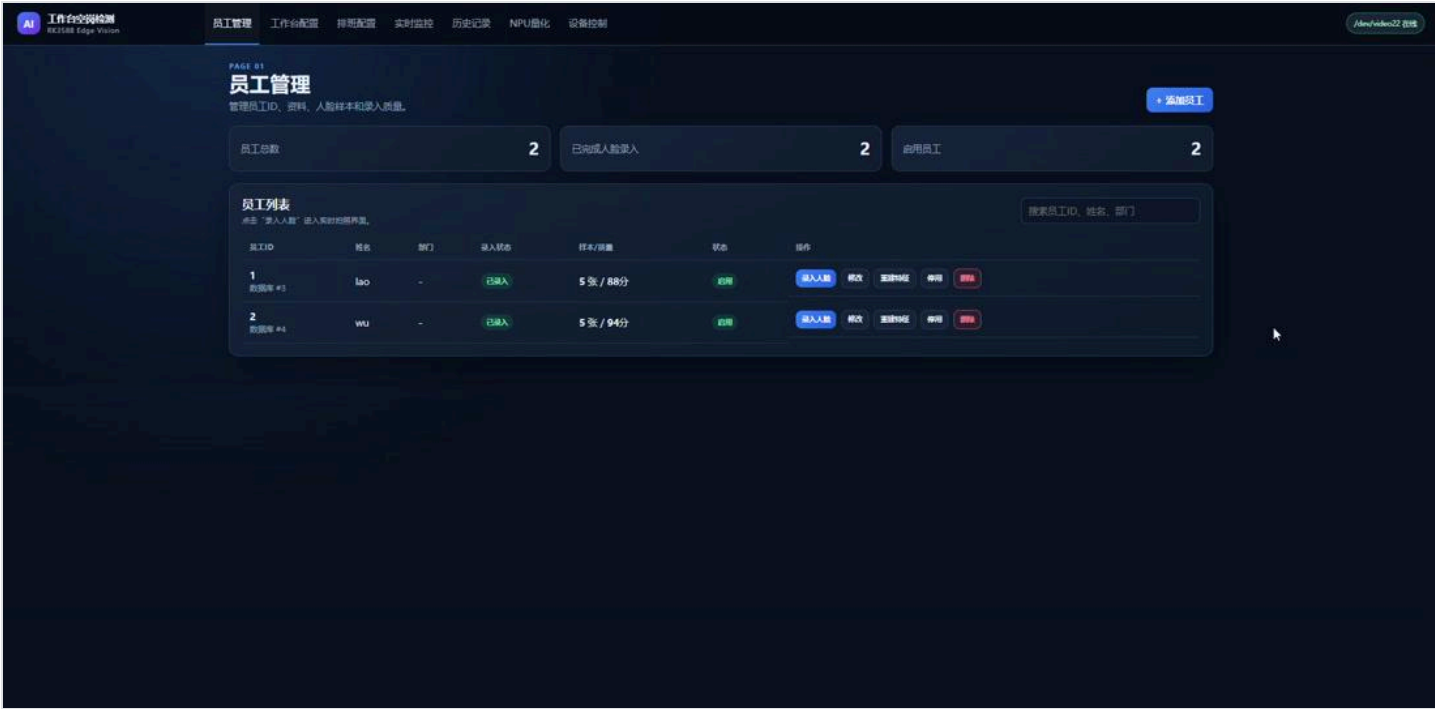
代码块

```
1 cd /root/workstation_absence_v1.0.6  
2 export PYTHONPATH=/root/workstation_absence_v1.0.6  
3 /root/yolov8_rk3588/rknn310/bin/python \  
4 /root/workstation_absence_v1.0.6/app.py \  
5 --config /root/workstation_absence_v1.0.6/config.json
```



1.7v

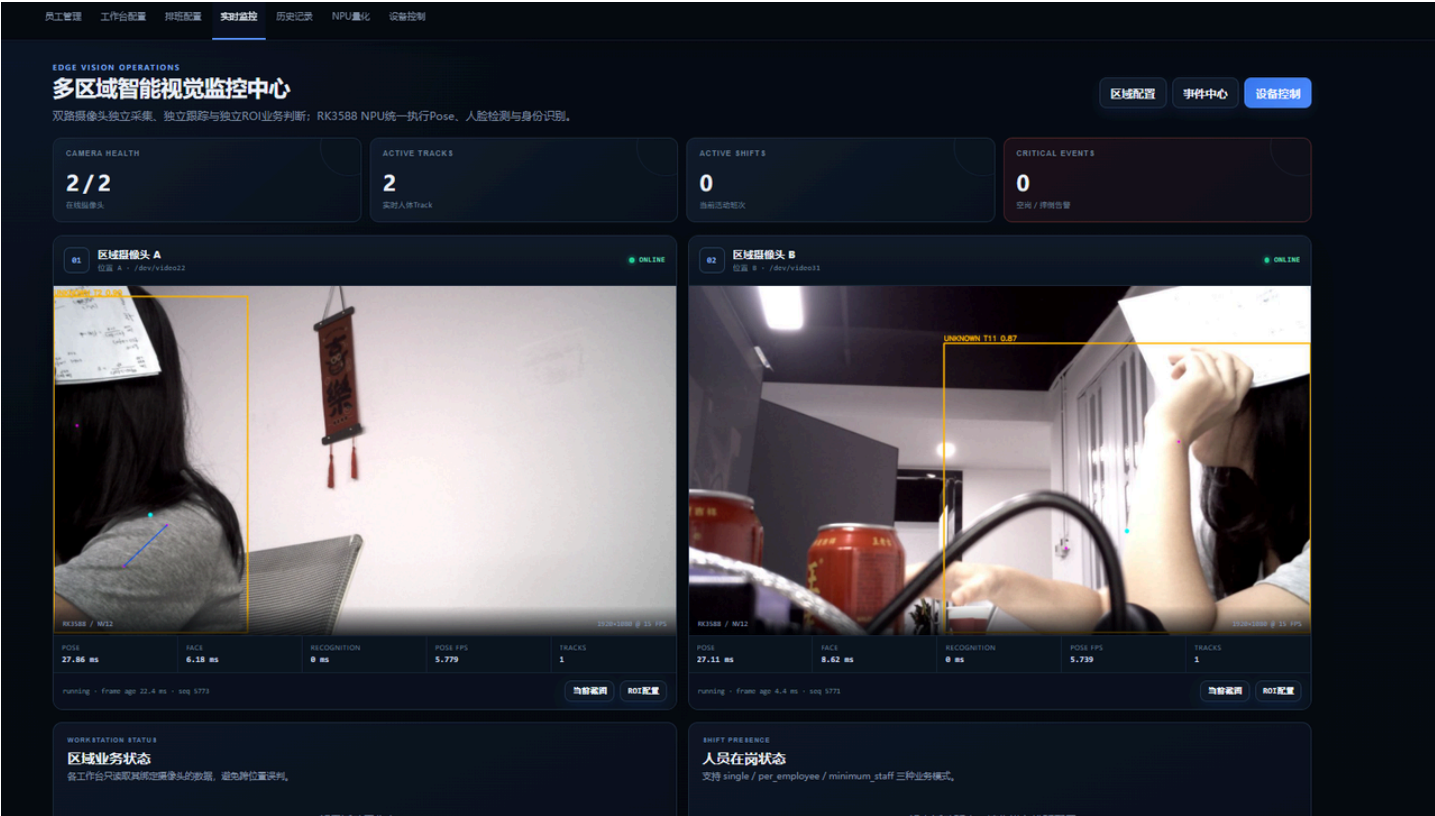
输入worker17



1.8v

cam1 → /dev/video22

cam2 → /dev/video31



结果分析



当前RK3588端已经成功加载三个INT8 RKNN模型，并分别分配到NPU Core 0、1和2。YOLOv8-Pose纯NPU平均推理延迟约25.99 ms，理论吞吐量约38.48 FPS，但完整项目实际按约8.59 FPS调度；RetinaFace纯推理约6 ms，项目按2.64 FPS降频运行；W600K-MBF纯推理约2.74 ms，只在需要确认员工身份时调用，因此实际频率为0.61 FPS。当前网页预览约8.58 FPS，与Pose更新频率基本一致，说明主链路节拍由Pose驱动

[PROFILE] 已采样 60.0s, 写报告并停止.....

流水线性能统计

stage	count	calls/s	avg ms	P50	P95	max
bytetrack_ms	1005	16.41	0.104	0.156	0.185	0.607
camera_nv12_to_bgr_ms	1756	28.68	3.690	1.951	14.434	31.775
face_embedding_color_convert_ms	33	0.54	0.030	0.027	0.047	0.066
face_embedding_inference_ms	33	0.54	2.734	2.603	3.258	4.060
jpeg_encode_ms	1004	16.40	18.921	18.907	19.330	22.277
pose_color_convert_ms	503	8.22	0.251	0.190	0.603	1.442
pose_inference_ms	503	8.22	25.531	25.184	27.624	56.206
retinaface_color_convert_ms	162	2.65	0.239	0.132	0.573	6.474
retinaface_inference_ms	162	2.65	6.080	5.824	7.159	24.323

核心结果可以概括为：

摄像头链路接近30 FPS，Pose主业务链约8.22 FPS；Pose纯NPU推理约25.5 ms，不是主要问题。当前明显耗时项是JPEG编码，同时还有约一半以上的主循环时间尚未被脚本捕获。

阶段	实际频率	平均耗时	P95	判断
NV12→BGR	28.68次/秒	3.690 ms	14.434 ms	平均快，但偶尔抖动明显
Pose推理	8.22次/秒	25.531 ms	27.624 ms	NPU比较稳定
ByteTrack	16.41次/秒	0.104 ms	0.185 ms	几乎可以忽略
RetinaFace推理	2.65次/秒	6.080 ms	7.159 ms	很快，采用降频调用
MobileFaceNet推理	0.54次/秒	2.734 ms	3.258 ms	很快，按需调用
JPEG编码	16.40次/秒	18.921 ms	19.330 ms	当前明显CPU耗时项

优化

异端网外

参考：<https://www.bilibili.com/video/BV1Wh411A73b?t=28.0>

```

keyboardinterrupt.
(rknn310) root@RK3588-Tronlong:~/workstation_absence# ^C
(rknn310) root@RK3588-Tronlong:~/workstation_absence# uname -m
aarch64
(rknn310) root@RK3588-Tronlong:~/workstation_absence# ls -l /dev/net/tun
ls: cannot access '/dev/net/tun': No such file or directory
(rknn310) root@RK3588-Tronlong:~/workstation_absence# mkdir -p /dev/net && mknod /dev/net/tun c 10 200 && chmod 666 /dev/net/tun
(rknn310) root@RK3588-Tronlong:~/workstation_absence# ls -l /dev/net/tun
crw-rw-rw- 1 root root 10, 200 Jul 30 17:40 /dev/net/tun
(rknn310) root@RK3588-Tronlong:~/workstation_absence# test -c /dev/net/tun && echo "TUN设备正常" || echo "TUN设备不可用"
TUN设备正常
(rknn310) root@RK3588-Tronlong:~/workstation_absence# |

```

Fpga

```

IMX219
↓ MIPI CSI
RK3588 ISP
↓ NV12
CameraService
↓
FrameSlot
↓
RGA预处理
↓
RKNN NPU Pose推理
↓
CPU后处理 + ByteTrack + ROI状态机
↓
MPP JPEG/H.264
↓
网页/手机端

```

方案A：NV12预处理流水线

FPGA处理内容

NV12输入
→ YUV转RGB
→ resize
→ letterbox
→ padding
→ uint8/int8格式转换
→ 输出RKNN输入尺寸



例如：

1920×1080 NV12
→ 640×640 RGB
→ NHWC uint8



FPGA内部模块

AXI4-Stream输入
→ NV12色度上采样
→ YUV2RGB定点矩阵
→ 双线性缩放
→ letterbox边界填充
→ 像素格式打包
→ AXI DMA输出



方案D：零拷贝DMA和帧队列

FPGA通过PCIe或其他高速接口连接RK3588时，必须避免：

```
FPGA DDR
→ CPU缓冲区
→ OpenCV Mat
→ 再复制到RKNN缓冲区
```



正确方式：

```
FPGA AXI DMA
→ RK3588连接物理内存/CMA
→ dnabuf
→ RGA/RKNN直接读取
```



每个DMA描述符带：

```
sequence
capture_ts_ns
width
height
stride
format
buffer_index
```



推荐环形队列：

```
buffer0
buffer1
buffer2
buffer3
```



生产者只写空闲buffer，消费者只读取已完成buffer。

如果消费者处理不过来，继续保留最新帧：

```
旧帧可覆盖
但sequence必须连续递增
overwrite_count必须记录
```



这与现在的FrameSlot设计完全一致。



位置1: IMX219直接进入FPGA

IMX219 → FPGA MIPI RX → 图像处理 → RK3588



优点:

- 可做真正的硬件同步
- 可以在像素流阶段完成预处理
- 最低理论延迟

缺点:

- 需要MIPI D-PHY RX
- 需要处理IMX219寄存器控制
- 可能绕过RK3588 ISP
- AE/AWB、去噪、曝光控制都要重新处理
- 驱动复杂
- 硬件风险最高

不推荐当前阶段直接采用。

位置2: RK3588采集后通过PCIe发送FPGA

IMX219 → RK3588 ISP → NV12 → PCIe DMA → FPGA



优点:

- 保留RKISP
- 软件和硬件边界清晰
- 容易调试
- 适合双目视差、滤波和复杂预处理

缺点:

- 图像需要传到FPGA
- 结果还要传回来
- 只有简单颜色转换时可能不划算

这是更适合当前项目的FPGA接入方案。

P0: 检查为什么每轮Pose做两次JPEG编码

项目阶段	FPGA适合程度
NV12→RGB/BGR	高
Resize/Letterbox	高
图像滤波、裁剪	高
JPEG编码	可以，但通常专用编码器更划算
YOLO推理	可以，需要DPU或自定义加速器
Pose后处理/NMS	可以，但开发复杂
ByteTrack	不值得，当前只有0.104 ms
ROI判断	不值得，通常远低于1 ms
FastAPI/WebSocket	不适合

```
(rknn310) root@RK3588-Tronlong:~/workstation_absence# source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && python -m json.tool reports/trace_image_calls.json | sed -n '1,260p'
```

```
{
  "elapsed_sec": 27.91661771700001,
  "jpeg_encode": [
    {
      "location": "/root/workstation_absence/workstation/runtime.py:346:_encode",
      "count": 358,
      "calls_per_sec": 12.82390308271455,
      "average_ms": 19.352147044703454,
      "p50_ms": 18.97528000017701,
      "p95_ms": 22.565571599739084,
      "max_ms": 24.131265000050917,
      "total_ms": 6928.068642003836
    }
  ],
  "nv12_to_bgr": [
    {
      "location": "/root/workstation_absence/workstation/camera.py:161:run",
      "count": 755,
      "calls_per_sec": 27.044823540361687,
      "average_ms": 3.135175173517099,
      "p50_ms": 1.897891999760759,
      "p95_ms": 14.01427780024278,
      "max_ms": 15.492897000058292,
      "total_ms": 2367.0572560054097
    }
  ]
}
```

位置:
/root/workstation_absence/workstation
/runtime.py:346:_encode

调用次数: 358
调用频率: 12.824次/秒
平均耗时: 19.352 ms
P50: 18.975 ms
P95: 22.566 ms
最大值: 24.131 ms
累计耗时: 6928.069 ms

```
(rknn310) root@RK3588-Tronlong:~/workstation_absence# source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && python -m json.tool reports/trace_image_calls.json | sed -n '1,260p'
```

```
{
  "elapsed_sec": 27.91661771700001,
  "jpeg_encode": [
    {
      "location": "/root/workstation_absence/workstation/runtime.py:346:_encode",
      "count": 358,
      "calls_per_sec": 12.82390308271455,
      "average_ms": 19.352147044703454,
      "p50_ms": 18.97528000017701,
      "p95_ms": 22.565571599739084,
      "max_ms": 24.131265000050917,
      "total_ms": 6928.068642003836
    }
  ],
  "nv12_to_bgr": [
    {
      "location": "/root/workstation_absence/workstation/camera.py:161:run",
      "count": 755,
      "calls_per_sec": 27.044823540361687,
      "average_ms": 3.135175173517099,
      "p50_ms": 1.897891999760759,
      "p95_ms": 14.01427780024278,
      "max_ms": 15.492897000058292,
      "total_ms": 2367.0572560054097
    }
  ]
}
```

位置:
/root/workstation_absence/workstation
/camera.py:161:run

调用次数: 755
调用频率: 27.044次/秒
平均耗时: 3.135 ms
P50: 1.898 ms
P95: 14.015 ms
最大值: 15.493 ms
累计耗时: 2367.057 ms

IMX219
↓ MIPI CSI
RK3588 ISP
↓ NV12图像
RK3588内存
↓ PCIe DMA
FPGA DDR
↓ FPGA图像处理
FPGA DDR
↓ PCIe DMA
RK3588内存
↓
RKNN NPU Pose推理

你上传的ARM+FPGA手册明确给出两种方向：

写操作：

ARM DDR → PCIe → FPGA DDR3



读操作：

FPGA DDR3 → PCIe → ARM DDR

因此从通信能力上，位置2可以实现。

 2-11-ARM + FPGA通信案例开发手册

3. 位置2为什么不适合当前项目

1080P NV12一帧大小约为：

```
1920 × 1080 × 1.5  
≈ 3.11 MB
```

30 FPS单向传输：

```
3.11 × 30  
≈ 93.3 MB/s
```

发送到FPGA，再返回RK3588：

```
约186.6 MB/s
```

带宽上通常可行，但会增加：

```
RK3588申请连续内存  
PCIe DMA驱动  
FPGA DDR帧缓存  
PCIe发送  
FPGA处理  
PCIe返回  
缓存同步  
sequence同步  
中断处理  
额外帧延迟
```

而当前只需要解决：

```
NV12→BGR P95抖动  
resize/letterbox  
JPEG编码19ms
```

为了避免位置1、位置2造成误解，把当前项目称为“位置0”。

```
IMX219  
↓  
RK3588 MIPI CSI  
↓  
RK3588 ISP  
↓ NV12  
CameraService  
↓  
FrameSlot  
├─ RGA: NV12转RGB、resize  
├─ NPU: YOLOv8-Pose  
└─ MPP: JPEG/H.264编码
```

位置0

一帧图像如何走完整条链路

1. IMX219把光线转换为原始图像信号，经MIPI CSI送入RK3588。
2. RK3588 ISP完成曝光、白平衡、去噪等处理，输出1920×1080 NV12。
3. CameraService通过v4l2-ctl持续读取固定大小的NV12字节。
4. OpenCV把NV12转换为BGR，写入FrameSlot，并为这帧记录sequence与capture_ts_ns。
5. RuntimeService只在sequence变化时复制一份最新帧，避免重复处理同一帧。
6. 按设定频率运行Pose、人脸识别和网页预览；不同任务互不要求同频。
7. Pose输出人体框和17个关键点；ByteTrackLite给每个人分配稳定Track ID。
8. RetinaFace找到人脸，W600K提取特征向量，并与员工库做相似度匹配。
9. PresenceManager与FallDetector根据Track、关键点和持续时间更新业务状态。
10. 最后绘制框、骨架、ROI和文字，压缩为JPEG，由FastAPI网页发送给浏览器

模块	通俗理解	worker17 中的用途
ISP	洗照片	IMX219 RAW → 正常 NV12
RGA	裁剪、缩放、转换格式	给 Pose 生成 640×640模型输入
MPP	压缩机	NV12 → JPEG
NPU	看图识别	YOLOv8-Pose、人脸推理

核心模块逐个解释

3.1 CameraService：负责稳定拿到图像

CameraService不是AI模型。它的工作是持续从V4L2设备节点读取NV12帧，并保证每次读到的字节数量正确。当前代码通过子进程运行v4l2-ctl，标准输出就是连续帧数据。

摄像头采集命令的核心形式

```
v4l2-ctl -d /dev/videoXX \  
--set-fmt-video=width=1920,height=1080,pixelformat=NV12 \  
--set-parm=30 --stream-mmap=4 --stream-poll --stream-to=
```

参数	意思	错误时表现
device	摄像头节点，例如/dev/video22	节点选错：无画面或格式不支持
width/height	每帧分辨率	字节数不匹配、图像错位
pixel_format=NV12	Y平面 + UV交错平面	颜色转换错误会花屏或偏色
fps=30	摄像头目标帧率	实际采集FPS低时要查驱动/曝光/带宽
buffers=4	V4L2 mmap缓冲数	太小可能抖动，太大可能增加积压

3.2 NV12、BGR、RGB到底是什么

格式	谁使用	特点
NV12	摄像头/ISP/MPP/RGA常见	1.5字节/像素，1080P约3.11 MB；节省带宽
BGR	OpenCV默认	3字节/像素，适合绘制和cv2处理
RGB	多数深度学习模型输入	通道顺序与BGR相反，顺序错会影响识别
BGRA/BGRx	某些硬件/插件接口	4字节/像素，需要正确stride和格式协商

3.3人脸检测与人脸识别不是一回事

模块	回答的问题	输出
RetinaFace	脸在哪里？五官点在哪里？	人脸框、5点关键点、置信度
5点对齐	把歪斜的人脸摆正	标准化112×112人脸

W600K/MobileFaceNet	这张脸的特征是什么？	归一化embedding向量
1:N匹配	最像员工库里的谁？	员工ID与余弦相似度

3.4JPEG预览：为什么它会影响Pose

网页不能直接显示NumPy的BGR数组，系统需要绘制、resize、JPEG编码再发送。软件JPEG编码会占用CPU和内存带宽，因此即使NPU推理时间不变，也可能让frame_age、跳帧和整体FPS变差。

对比项	JPEG开启	JPEG关闭	解释
进程CPU平均	约238%	约161%	关闭后约释放0.78个CPU核心
Pose实际FPS	约7.69	约8.60	整体调度更轻
Frame age P95	约41.62 ms	约34.78 ms	消费者拿到的帧更“新”
NV12→BGR P95	约13.89 ms	约14.02 ms	尾部抖动不是JPEG单独造成
overwrite_ratio	约74.3%	约11.2%	JPEG会加剧处理线程落后

MPP当前结论

MPP独立NV12编码能力已验证：1280×720约7.2 ms，明显快于OpenCV JPEG约19.2 ms；但标注BGR/BGRA接入后出现颜色与细节失真，因此目前稳定版本仍用OpenCV单路8 FPS。不要把“mpp_calls>0”当成画面质量验收。

错误链路

BGR

→ BGRA

→ GStreamer videoconvert

→ NV12

→ MPP JPEG

中间可能发生：

- BGRA/BGRx/RGBA解释不一致；
- stride错误；
- buffer长度错误；
- BT.601/BT.709不一致；
- UV平面偏移错误；

- GStreamer插件对BGRA路径兼容性不足。

MPP负责“压缩”，SIMD负责“把数据整理成MPP可靠接受的NV12”；最终正式方案应升级为RGA负责转换、MPP负责JPEG。

位置2

问题

batch维度不对

```
Show or set hostname or DNS domain name

-s      Short
-i      Addresses for the hostname
-d      DNS domain name
-f      Fully qualified domain name
-F FILE Use FILE's content as hostname
http://板子IP:8080/

按 Ctrl+C 停止程序。
E Catch exception when setting inputs.
E Traceback (most recent call last):
  File "/root/yolov8_rk3588/rknn310/lib/python3.10/site-packages/rknnlite/api/rknn_lite.py", line 209, in inference
    self.rknn_runtime.set_inputs(inputs, data_type, data_format, inputs_pass_through=inputs_pass_through)
  File "rknnlite/api/rknn_runtime.py", line 1082, in rknnlite.api.rknn_runtime.RKNNRuntime.set_inputs
Exception: The input[0] need 4dims input, but 3dims input buffer feed.

===== RKNN Pose 模型输出 =====
Pose 推理线程失败: TypeError: object of type 'NoneType' has no len()

正在停止双摄 Pose 程序.....
双摄采集已停止
RKNN NPU 资源已释放
结果目录: /root/yolov8_rk3588/result_dual_pose
(rknn310) root@RK3588-TronLong:~/yolov8_rk3588# |
```

一、错误原因

这次不是模型路径问题，而是输入张量少了一个 batch 维度。

报错已经明确：

```
The input[0] need 4dims input, but 3dims input buffer feed.
```

旧代码传入的是：

```
(640, 640, 3)
```

RKNNLite要求：

```
(1, 640, 640, 3)
```

即：

```
<? Python
```

```
input_tensor = np.expand_dims(input_rgb, axis=0)
outputs = rknn.inference(inputs=[input_tensor])
```

第一次推理失败后，`outputs` 变成了 `None`，旧代码又执行：

```
<? Python
```

```
len(outputs)
```

所以才出现第二个报错：

```
TypeError: object of type 'NoneType' has no len()
```

你原来的双摄脚本本身已经定义了 `--model` 参数，模型路径可以从命令行覆盖，不需要修改源代码。 `dual_yolov8_rknn_camera`

No module named 'workstation'

```
(rknn310) root@RK3588-Tronlong:~/workstation_absence# source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && python tools/check_environment.py --config config.json
Traceback (most recent call last):
  File "/root/workstation_absence/tools/check_environment.py", line 10, in <module>
    from workstation.camera import CameraManager
ModuleNotFoundError: No module named 'workstation'
(rknn310) root@RK3588-Tronlong:~/workstation_absence#
```

示工作台：

工具脚本直接运行时没有把项目根目录加入 Python 搜索路径

```
source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && export
PYTHONPATH=/root/workstation_absence && python tools/check_environment.py --config
config.json
```

系统 Python 没有 SQLite

```
ModuleNotFoundError: No module named 'sqlite3'
(rknn310) root@RK3588-Tronlong:~/workstation_absence# /usr/bin/python3 -c "import sys,sqlite3; print('Python:',sys.version); print('SQLite:',sqlite3.sqlite_version)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'sqlite3'
(rknn310) root@RK3588-Tronlong:~/workstation_absence# cat /etc/os-release
NAME=Buildroot
VERSION=linux-5.10-gen-rkr8-37-gea3737bd41-dirty
ID=buildroot gea3737b
VERSION_ID=2021.11
PRETTY_NAME="Buildroot 2021.11"
ID_LIKE="unknown"
BUILD_TIMESTAMP="tronlong@tronlong-virtual-machine Fri May 16 09:38:59 CST 2025"
```

为了不重编整个板端固件，我已经把项目数据库替换为**纯Python JSON持久化后端**，不再依赖

sqlite3

Buildroot 精简版 Python 的日志模块无法识别 Uvicorn 默认配置中的字符串 "INFO"

```
(rknn310) root@RK3588-Tronlong:~/workstation_absence# source /root/yolov8_rk3588/rknn310/bin/activate && cd /root/workstation_absence && export PYTHONPATH=/root/workstation_absence && python app.py --config config.json
I RKNN: [14:43:27.713] RKNN Runtime Information, librknrt version: 2.0.0b0 (35a6907d79@2024-03-24T10:31:14)
I RKNN: [14:43:27.713] RKNN Driver Information, version: 0.9.6
I RKNN: [14:43:27.714] RKNN Model Information, version: 6, toolkit version: 2.3.2(compiler version: 2.3.2 (e045de294f@2025-04-07T19:48:25)), target: RKNP
U v2, target platform: rk3588, framework name: ONNX, framework layout: NCHW, model inference type: static_shape
W RKNN: [14:43:27.714] RKNN Model version: 2.3.2 not match with rknn runtime version: 2.0.0
W RKNN: [14:43:27.725] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn model is static shape type, please export rknn with dynamic_shapes
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please ignore the above warning message.)
I RKNN: [14:43:27.987] RKNN Runtime Information, librknrt version: 2.0.0b0 (35a6907d79@2024-03-24T10:31:14)
I RKNN: [14:43:27.987] RKNN Driver Information, version: 0.9.6
I RKNN: [14:43:27.987] RKNN Model Information, version: 6, toolkit version: 2.3.2(compiler version: 2.3.2 (e045de294f@2025-04-07T19:48:25)), target: RKNP
U v2, target platform: rk3588, framework name: ONNX, framework layout: NCHW, model inference type: static_shape
W RKNN: [14:43:27.987] RKNN Model version: 2.3.2 not match with rknn runtime version: 2.0.0
W RKNN: [14:43:27.930] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn model is static shape type, please export rknn with dynamic_shapes
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please ignore the above warning message.)
I RKNN: [14:43:28.092] RKNN Runtime Information, librknrt version: 2.0.0b0 (35a6907d79@2024-03-24T10:31:14)
I RKNN: [14:43:28.093] RKNN Driver Information, version: 0.9.6
I RKNN: [14:43:28.093] RKNN Model Information, version: 6, toolkit version: 2.3.2(compiler version: 2.3.2 (e045de294f@2025-04-07T19:48:25)), target: RKNP
U v2, target platform: rk3588, framework name: ONNX, framework layout: NCHW, model inference type: static_shape
W RKNN: [14:43:28.093] RKNN Model version: 2.3.2 not match with rknn runtime version: 2.0.0
W RKNN: [14:43:28.099] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn model is static shape type, please export rknn with dynamic_shapes
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please ignore the above warning message.)
Traceback (most recent call last):
  File "/usr/lib/python3.10/logging/config.py", line 619, in configure
    File "/usr/lib/python3.10/logging/config.py", line 796, in configure_logger
    File "/usr/lib/python3.10/logging/config.py", line 781, in common_logger_config
    File "/usr/lib/python3.10/logging/__init__.py", line 198, in _checkLevel
ValueError: Unknown level: 'INFO'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "/root/workstation_absence/app.py", line 64, in <module>
    main()
  File "/root/workstation_absence/app.py", line 55, in main
    uvicorn.run(
  File "/root/yolov8_rk3588/rknn310/Lib/python3.10/site-packages/uvicorn/main.py", line 551, in run
    config = Config(
  File "/root/yolov8_rk3588/rknn310/Lib/python3.10/site-packages/uvicorn/config.py", line 296, in __init__
    self.configure_logging()
  File "/root/yolov8_rk3588/rknn310/Lib/python3.10/site-packages/uvicorn/config.py", line 391, in configure_logging
    logging.config.dictConfig(self.log_config)
  File "/usr/lib/python3.10/logging/config.py", line 810, in dictConfig
    File "/usr/lib/python3.10/logging/config.py", line 621, in configure
ValueError: Unable to configure logger 'uvicorn'
(rknn310) root@RK3588-Tronlong:~/workstation_absence# |
```

代理时项目传不上github

```
C:\Users\PC\Desktop\workstation_absence>netstat -ano | findstr LISTENING | findstr /R ":7890 :7897 :10809 :10808 :1080"
TCP    0.0.0.0:7890          0.0.0.0:0          LISTENING        26492
TCP    [::]:7890           [::]:0             LISTENING        26492

C:\Users\PC\Desktop\workstation_absence>C:\Users\PC\Desktop\workstation_absence>netstat -ano | findstr LISTENING | findstr /R ":7890 :7897 :10809 :10808 :1080"
'C:\Users\PC\Desktop\workstation_absence' 不是内部或外部命令，也不是可运行的程序
或批处理文件。

C:\Users\PC\Desktop\workstation_absence> TCP    0.0.0.0:7890          0.0.0.0:0          LISTENING        26492
'TCP' 不是内部或外部命令，也不是可运行的程序
或批处理文件。

C:\Users\PC\Desktop\workstation_absence> TCP    [::]:7890           [::]:0             LISTENING        26492
'TCP' 不是内部或外部命令，也不是可运行的程序
或批处理文件。

C:\Users\PC\Desktop\workstation_absence>
C:\Users\PC\Desktop\workstation_absence>C:\Users\PC\Desktop\workstation_absence>
命令语法不正确。
C:\Users\PC\Desktop\workstation_absence>git config --global --get http.proxy
C:\Users\PC\Desktop\workstation_absence>git config --global --get https.proxy

C:\Users\PC\Desktop\workstation_absence>curl -I -x http://127.0.0.1:7897 https://github.com
curl: (7) Failed to connect to github.com:443 over proxy 127.0.0.1 after 2045 ms: Could not connect to server

C:\Users\PC\Desktop\workstation_absence>|
curl -I -x http://127.0.0.1:7897 https://github.com
```